Enforce least-privilege access through identity-aware, application-specific access policies using Zscaler Private Access (ZPA).

College name: Vidyasagar college of arts and science

College code: BRU30

Team Id: SWTID-2026-7448

Team members:

Team size: 5

Team leader name: Narmatha N

Email: narmatha7852@gmail.com

Team member: RENUGOPAL K

Email: harih72067@gmail.com

Team member: SANDHIYA M

Email: dhiyasandhiya2005@gmail.com

Team member: PRAVEEN KUMAR S

Email: praveensakkarpani2006@gmail.com

Team member: MUTHU KARTHIK M

Email: karthikarthi79367@gmail.com

INTRODUCTION:

In an era where the digital landscape is constantly evolving, organizations face an ever-increasing threat from cyberattacks. The need for swift and effective incident response has become paramount to safeguard sensitive data, protect digital assets, and maintain the trust of stakeholders. In response to this growing challenge, we present a visionary project: the development of an AI-driven incident response platform that empowers cybersecurity teams with the tools to automate and enhance incident triage and response tasks. In response to the evolving threat landscape and the dissolution of the traditional network perimeter, modern enterprises are rapidly shifting away from legacy VPN-centric remote access models. These conventional approaches often grant excessive network-level access, creating a significant attack surface and increasing the risk of lateral threat movement. To address these challenges, this project centers on the implementation of a Zero Trust security architecture, fundamentally operating on the principle of "never trust, always verify." By moving beyond location-based access, we aim to establish a security posture where trust is no longer implicit, and access is determined by granular context rather than network connectivity. The core of this initiative involves the deployment of Zscaler Private Access (ZPA), a cloud-delivered Zero Trust Access solution designed to create identity-aware, application-specific connectivity. Unlike traditional VPNs that place users on the internal network, ZPA leverages a "service-initiated" model to establish direct, one-to-one connections between authorized users and specific applications, completely hiding the corporate network and IP address space from the internet. This ensures that users connect exclusively to the applications they are permitted to access, based on continuous identity verification and context, rendering the applications invisible to unauthorized users and preventing common attack vectors such as port scanning and distributed denial-of-service (DDoS) attacks. Ultimately, this project will enforce a least-privilege access model, ensuring users and devices are granted the minimum required access necessary to perform their functions. By decoupling application access from network access, we will eliminate the risk of lateral movement across the network, as users have no visibility or connectivity to other hosts. This strategic move not only significantly enhances security by reducing the attack surface but also simplifies the user experience and reduces operational overhead associated with legacy remote access infrastructure, positioning the organization at the forefront of secure and efficient access management.

ABSTRACT:

The proliferation of hybrid work and cloud-based resources has rendered traditional perimeter-based security models, such as VPNs, obsolete and increasingly dangerous. These legacy solutions often provide overly broad network access, exposing organizations to significant risks including lateral threat movement and an expanded attack surface. This project addresses these vulnerabilities by implementing a Zero Trust Architecture (ZTA) utilizing Zscaler Private Access (ZPA). The primary objective is to transition from network-centric to identity-centric access, thereby enforcing the principle of least privilege. The methodology involves deploying ZPA's cloud-based service to create application-specific, identityaware micro-tunnels. Unlike conventional VPNs that place users on the internal network, this solution establishes direct, one-to-one connections between verified users and authorized applications. The network infrastructure and all non-authorized applications remain completely invisible to the end-user, effectively eliminating surface for reconnaissance and attack. Access policies are dynamically governed by user identity, device posture, and contextual attributes, ensuring that connectivity is strictly limited to what is explicitly required for specific job functions. The implementation of ZPA successfully decouples application access from network access, resulting in a demonstrable reduction in the organization's attack surface and the complete prevention of lateral movement. This project delivers a robust, scalable, and user-friendly remote access framework that aligns with modern security best practices. By shifting to an identity-aware, least-privilege model, the organization significantly enhances its security posture while simultaneously reducing operational complexity and improving the end-user experience.

Use case scenario

1. Finance App Access – Only Finance Department

Use Case Scenario
A company hosts an internal **Finance ERP application** in a private data center. Only users in the **Finance department** should access it.

Policy Configuration

- **Application Segment:** Finance ERP
- **User Group:** Finance Users (from Identity Provider)

- **Device Posture:** Managed corporate devices only
- **Location:** Any

Output / Result

- Finance employees can securely access the ERP system through ZPA.
- Employees from other departments receive **access denied**.
- No direct network exposure to the ERP server.

---

2. Developer Access – Restricted to Corporate Devices

Use Case Scenario
Developers need access to **internal Git servers and build environments**, but access must be allowed **only from company-managed laptops**.

Policy Configuration

- **Application Segment:** DevOps Tools
- **User Group:** Developers
- **Device Posture:** Device must be compliant (MDM enrolled)
- **Client Type:** Zscaler Client Connector installed

Output / Result

- Developers using corporate laptops can access Git and build servers.
- If they try from personal devices → **access blocked**.
- Protects source code from being accessed via unmanaged systems.

---

3. Third-Party Vendor Access – Limited Application Access

Use Case Scenario
A third-party vendor needs temporary access to a **ticketing portal** hosted internally.

Policy Configuration

- **Application Segment:** Internal Ticketing Portal

- **User Group:** Vendor_Group
- **Access Time:** Business hours only
- **Device Type:** Browser access allowed

Output / Result

- Vendors can only access the **ticketing portal URL**.
- They cannot see or access any other internal applications.
- Access automatically stops outside business hours.

---

4. Admin Access – Multi-Factor Authentication Required

Use Case Scenario
System administrators access **critical infrastructure tools** like server management consoles.

Policy Configuration

- **Application Segment:** Admin Tools
- **User Group:** IT Admins
- **Authentication:** MFA required
- **Device Posture:** Managed device + secure posture

Output / Result

- Admin must complete **MFA verification** before accessing the application.
- Unauthorized users cannot reach the admin console.
- Provides **Zero Trust access control**.

---

5. Geographic Access Restriction

Use Case Scenario
A company wants internal HR systems accessed **only from employees working within India**.

Policy Configuration

- **Application Segment:** HR Portal
- **User Group:** Employees
- **Location:** India only
- **Authentication:** Standard SSO

Output / Result

- Employees accessing from India → **access allowed**.
- Login attempts from outside India → **blocked automatically**.
- Reduces risk of international credential misuse.

Stage – 1

Title: Enforce least-privilege access through identity-aware, application-specific access policies using Zscaler Private Access (ZPA).

Overview:

With the increasing sophistication of cyber threats and the gradual disappearance of the traditional network perimeter, modern organizations are moving away from legacy VPN-based remote access solutions. These traditional models typically provide broad network-level access, which expands the attack surface and increases the possibility of lateral movement by attackers within the network. To overcome these limitations, this project focuses on implementing a **Zero Trust security architecture**, which follows the principle of *"never trust, always verify."* Instead of relying on network location for trust, this model ensures that access decisions are based on identity, context, and continuous verification.

The main objective of this project is to deploy **Zscaler Private Access (ZPA)**, a cloud-delivered Zero Trust Network Access (ZTNA) solution that enables secure, identity-based connectivity to private applications. Unlike traditional VPNs that place users directly on the corporate network, ZPA uses a **service-initiated connection model** to create secure, one-to-one connections between authorized users and specific applications. This approach completely hides the internal network and IP address space from the public internet. As a result, users can only connect to the applications they are explicitly authorized to access, while unauthorized users cannot even detect the presence of these applications, thereby preventing threats such as port scanning and distributed denial-of-service (DDoS) attacks.

The implementation of this solution will enforce a **least-privilege access framework**, ensuring that users and devices receive only the minimum level of access required to perform their roles. By separating application access from network access, the solution eliminates the possibility of lateral movement within the network since users have no visibility into other systems or hosts. This transformation not only strengthens the organization's security posture by minimizing the attack **surface but also improves user experience and reduces the operational** complexity associated with maintaining traditional remote access infrastructure. Ultimately, this initiative positions the organization to adopt a more secure, scalable, and efficient access management strategy.

List of vulnerability

S. No	Vulnerability Name	CWE No	
1	Cross-site scripting (stored)	79	
2	Broken Access Control	285	
3	SQL Injection	89	
4	Broken Authentication	285	
5	Insecure Direct Object Reference	639	
6	Security Misconfiguration	732	
7	Cross-Site Request Forgery	352	
8	Cleartext Transmission ofSensitive Information	319	
9	Click Jacking	1021	

10	webserver allows password auto-completion	310	

Report:

Vulnerability name: Cross site scripting (stored)

CWE: 79 OWASP

category: A03:2021 -Injections

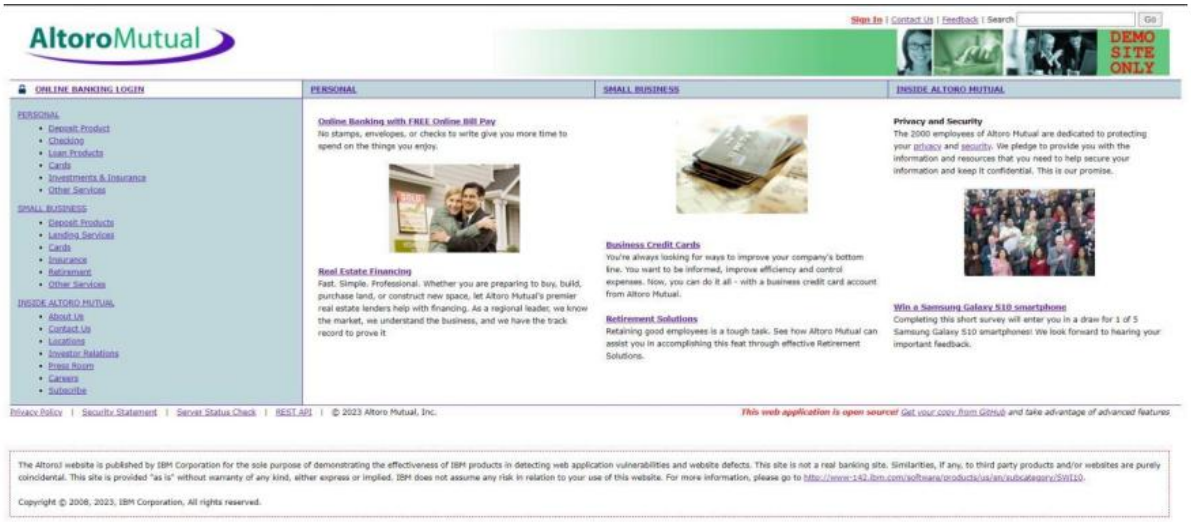
Description: It occurs when a malicious script is injected directly into a vulnerable web application. Reflected XSS involves the reflection of a malicious script of a web application, onto a user's browser. Business Impact: The application stores dangerous data in a database, message forum, visitor log, or other trusted data store. Later, the dangerous data is subsequently read back into the application and included in dynamic content. From an attacker's perspective, the optimal place to inject malicious content is in an area that is displayed to either many users or particularly interesting users. Interesting users typically have elevated privileges in the application or interact with sensitive data that is valuable to the attacker. If one of these users executes malicious content, the attacker may be able to perform privileged operations on behalf of the user or gain access to sensitive data belonging to the user.

Vulnerability path: http://testfire.net/

Vulnerabilityparameter:http://testfire.net/search.jsp?query=%27%3E%3Cscript %3Ealert%28%27hacked%27%29%3C%2Fscrip t%3E

Steps to Reproduce:

➢ Access the URL



In the search box we will input some code to perform the vulnerability.

The Script we will be inputting is '>

➢ This displays a harmless pop up alert box with the text saying ‘hacked’

Recommendation:

• When interacting with databases, use parameterized statements or prepared statements to avoid SQL injection, which can be a vector for XSS. • Sanitize and validate all user inputs and ensure that any data displayed on the web page

is properly encoded. Use output encoding libraries or functions to prevent script injection.

2. Vulnerability name: Broken access control

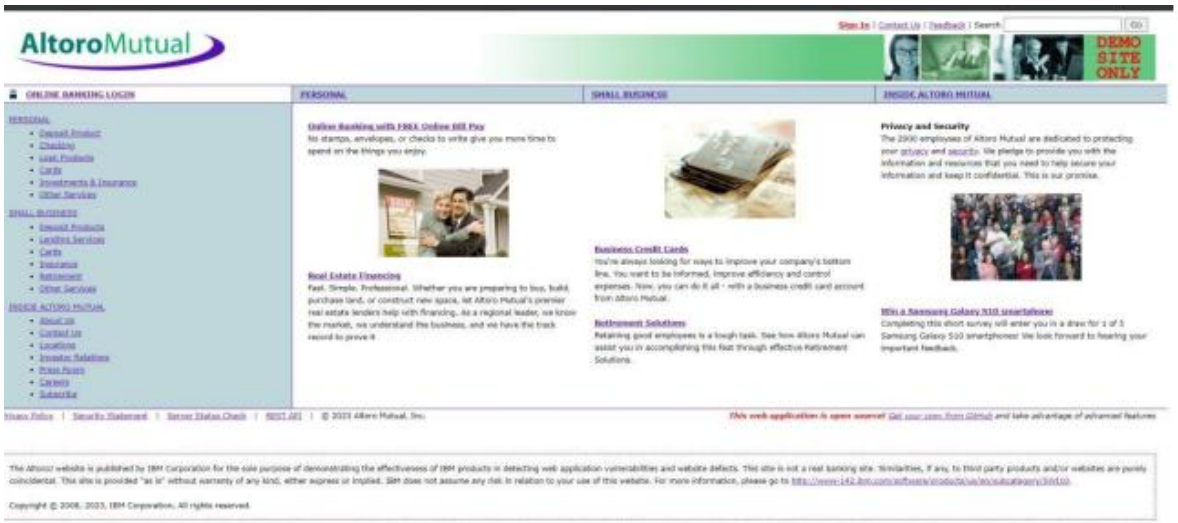
CWE: 285

Description: When access control checks are not applied consistently - or not at all - users are able to access data or perform actions that they should not be allowed to perform. This can lead to a wide range of problems, including information exposures, denial of service, and arbitrary code execution. Business Impact: wide-ranging business impact, including financial losses, reputation damage, legal consequences, operational disruption, and loss of customer trust. Addressing this vulnerability is crucial to protect the organization and its stakeholders from these potential negative effects.

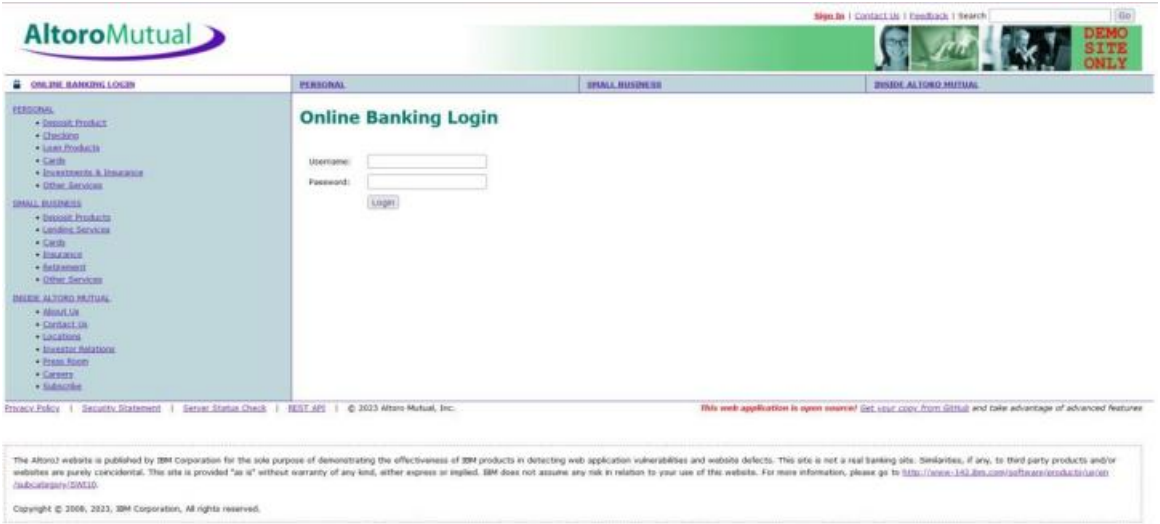
Vulnerability path: http://testfire.net/

Vulnerability parameter: http://testfire.net/login.jsp Steps to

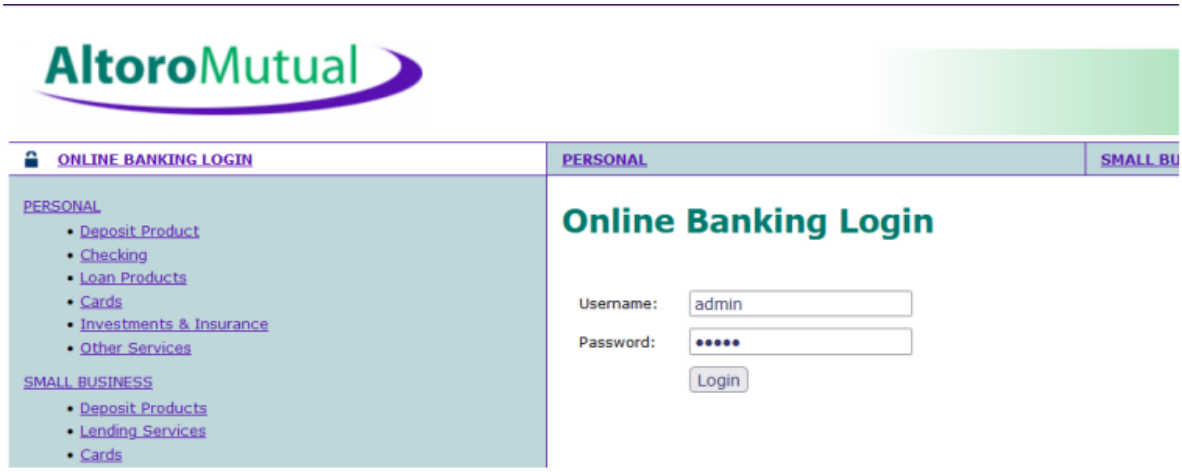
Reproduce: ➢ Access the URL



Now we will try to sign in to this website with admin privileges



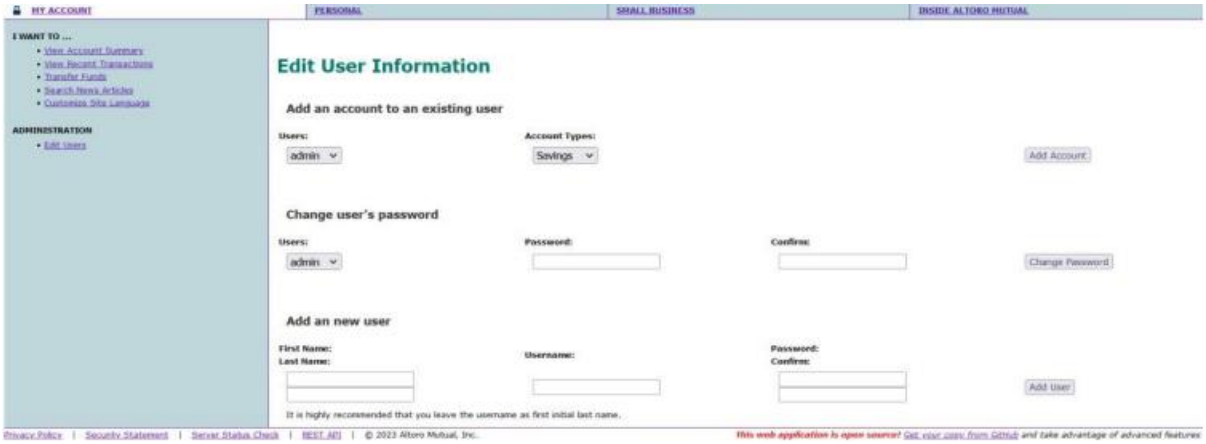
Also, we will be using burp suite to get requests from the website and know additional information. We use ‘admin’ for the username and password.



This request has been received in the burp suite with the username and password as well in clear text.

now we click on forward request in the burp suite and then we will be redirected to the admin user details.

We can even edit the other users data and delete their login details as well.



Recommendations:

• Ensure that the application has a robust authentication and authorization mechanism in place. Users should be authenticated before any sensitive actions are performed, and authorization checks should be conducted to verify that users have the necessary permissions for the requested operation.

• Ensure that session management is secure. Use strong session IDs, enforce session timeouts, and regenerate session tokens after login. Implement session fixation protection to prevent session hijacking.

3. Vulnerability name: SQL injection

CWE: 89

Description: The product constructs all or part of an SQL command using externally-influenced input from an upstream component, but it does not neutralize or incorrectly neutralizes special elements that could modify the

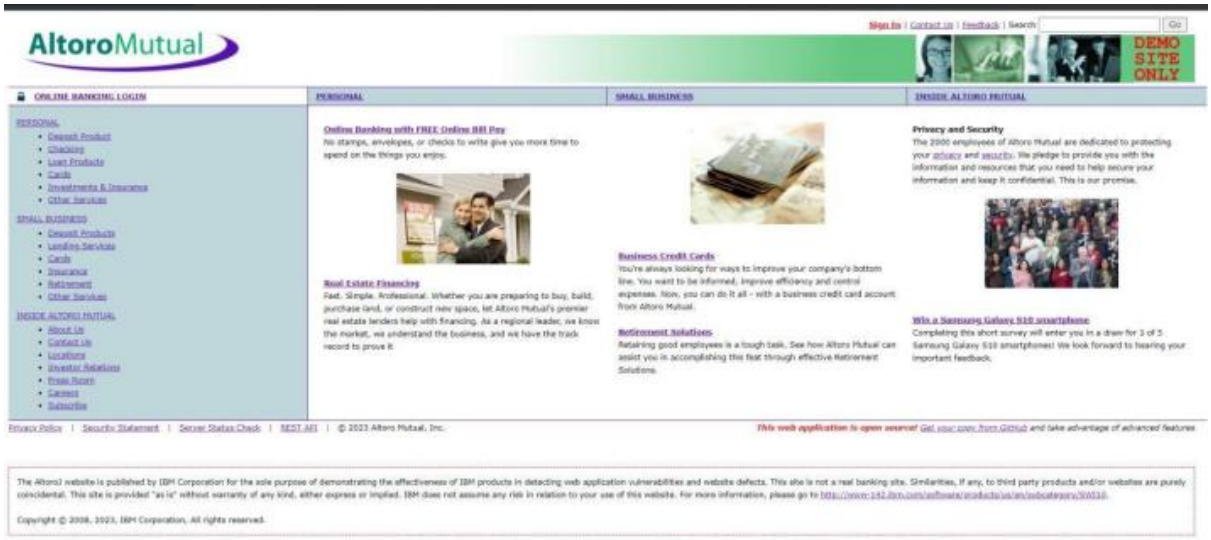
intended SQL command when it is sent to a downstream component. Business Impact: In summary, it is crucial to underscore that CWE-89, known as SQL Injection, can exert a profound and diverse business impact. This encompasses critical facets such as data breaches, financial setbacks, harm to reputation, legal ramifications, and operational turmoil. Hence, the imperative of preventing and remedying SQL injection vulnerabilities cannot be overstated, as it is indispensable for fortifying the security and continuity of an organization's applications and data.

Vulnerability path: http://testfire.net/

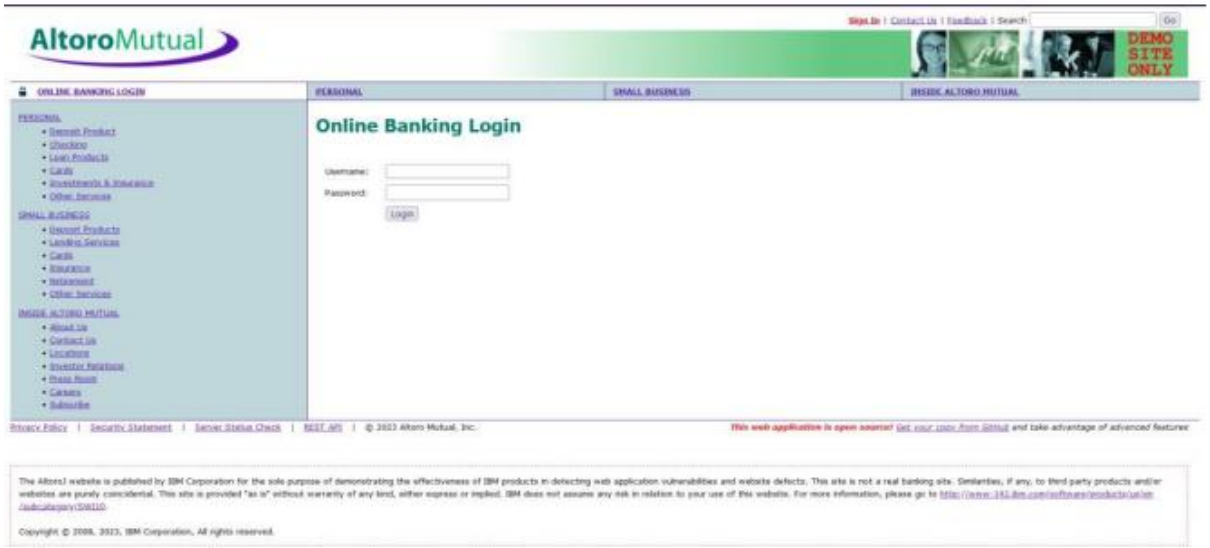
Vulnerability parameter: http://testfire.net/login.jsp

Steps to Reproduce:

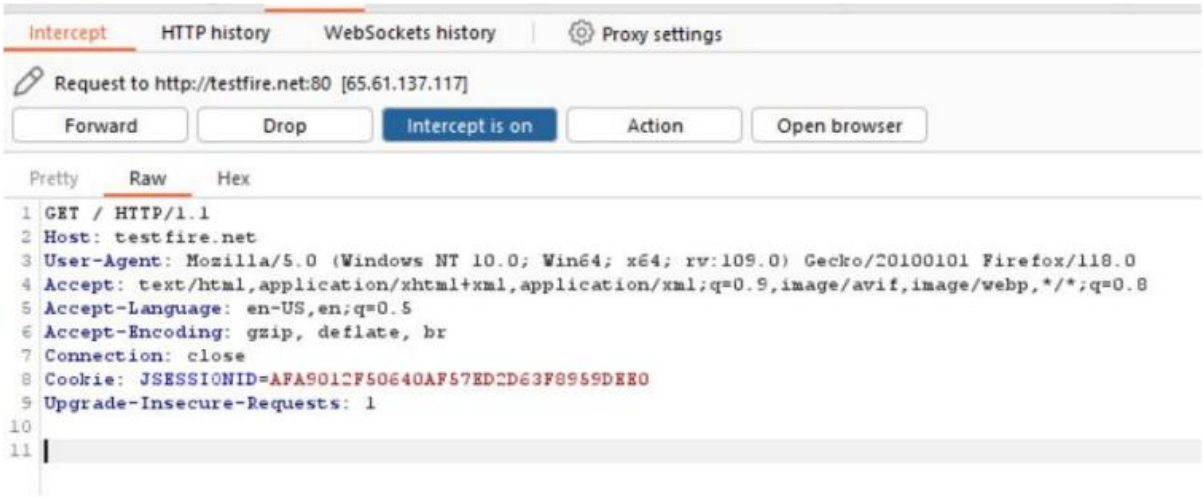
Access the URL

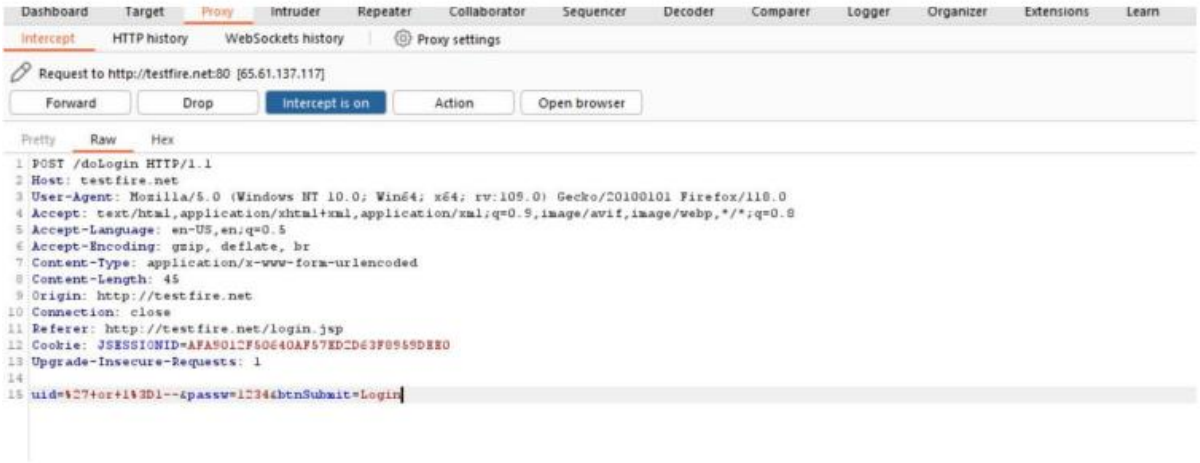


Now we will try to sign in to this website with admin privileges but using SQL injection



Also, we will be using burp suite to get requests from the website and know additional information.





Click forward request multiple times to proceed to the login page.

With this we can know that sql injection worked and we got the admin privileges

Recommendations:

• If parameterized statements are not feasible, use proper input validation and escaping mechanisms to sanitize user inputs before they are used in SQL queries. This helps prevent malicious code injection.

• Utilize stored procedures for database operations. This can help separate SQL code from application code and reduce the risk of SQL injection.

• Implement strict input validation for all user inputs, ensuring that data adheres to the expected format and structure.

4. Vulnerability name: Broken authentication

CWE: 285

Description: The system's authorization functionality does not prevent one user from gaining access to another user's data or record by modifying the key value identifying the data.

Business Impact: To effectively mitigate the business impact of CWE-285, it is imperative that organizations place a strong emphasis on bolstering their authentication and session management practices. This should encompass the adoption of multi-factor authentication, the secure storage of credentials, meticulous session handling, and a commitment to conducting routine security

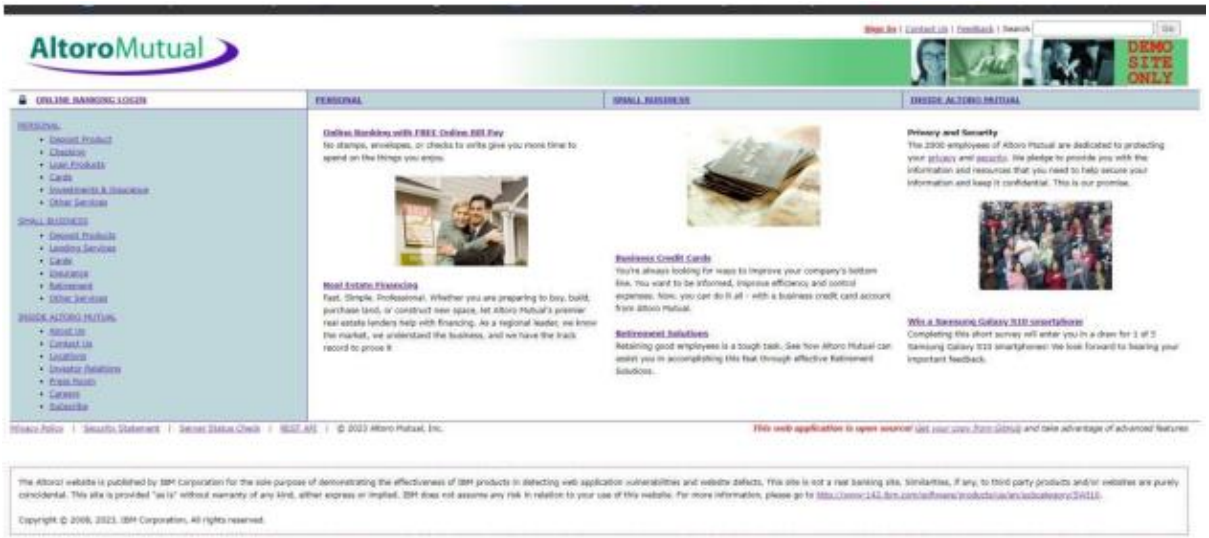
assessments and testing. The rectification of these vulnerabilities stands as a critical imperative, safeguarding sensitive data, user identities, and the organization's overarching security stature and reputation.

Vulnerability path: http://testfire.net/

Vulnerability parameter: http://testfire.net/login.jsp

Steps to Reproduce:

Access the URL



Now we will try to login using some different approach.

As we know which users are present in the database of this website by using the admin privileges. We can directly access a particular user by simply knowing their username; we will add some characters after his user name as a sql injection to simply bypass the password.

This leads us to the details of this person's account.

Recommendations:

• Implement strong authentication mechanisms, such as multi-factor authentication (MFA) or twofactor authentication (2FA), to ensure that only authorized users can access the system.

• Implement proper authorization controls to ensure that users have the necessary permissions to access specific functions or data. Use role-based access control (RBAC) to manage user privileges effectively.

• Deploy intrusion detection and prevention systems (IDS/IPS) to monitor and block suspicious activities related to authentication and authorization.

5. Vulnerability Name: Insecure Direct object Reference

CWE: 639

OWASP Category: A01: Broken Access Control

Description:

Insecure Direct Object Reference (IDOR) is a vulnerability that arises when attackers can access or modify objects by manipulating identifiers used in a web application's URLs or parameters. It occurs due to missing access control checks, which fail to verify whether a user should be allowed to access specific data.

Business Impact:

To effectively mitigate the business impact of CWE-639, organizations must make it a top priority to fortify their access control and authorization mechanisms. This entails the implementation of robust security measures, the regular conduct of comprehensive security assessments, and the deployment of intrusion detection systems to promptly identify and counter unauthorized access attempts. These measures stand as absolutely critical in the defense of sensitive data, the overall security of systems, and the preservation of the organization's esteemed reputation.

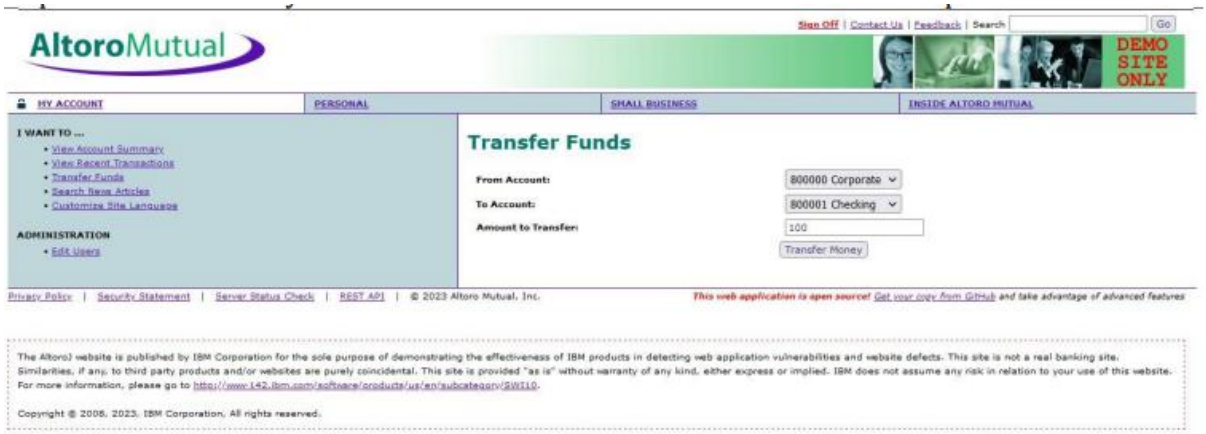
Vulnerability Path: http://testfire.net

Vulnerability Parameter: http://testfire.net/bank/transfer.jsp

Steps to Reproduce:

- ➢ Go to the domain.
- ➢ Sign into the site.

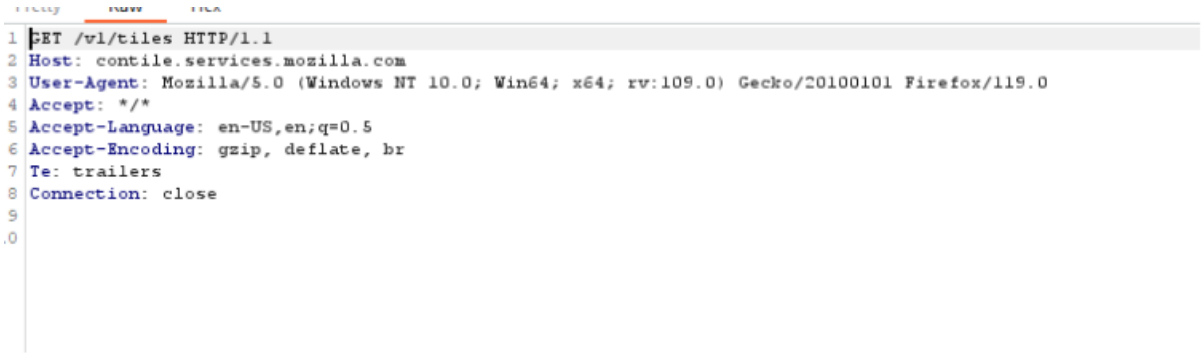
Open “Transfer Money” on the left side and fill in the details. On the intercept and click transfer



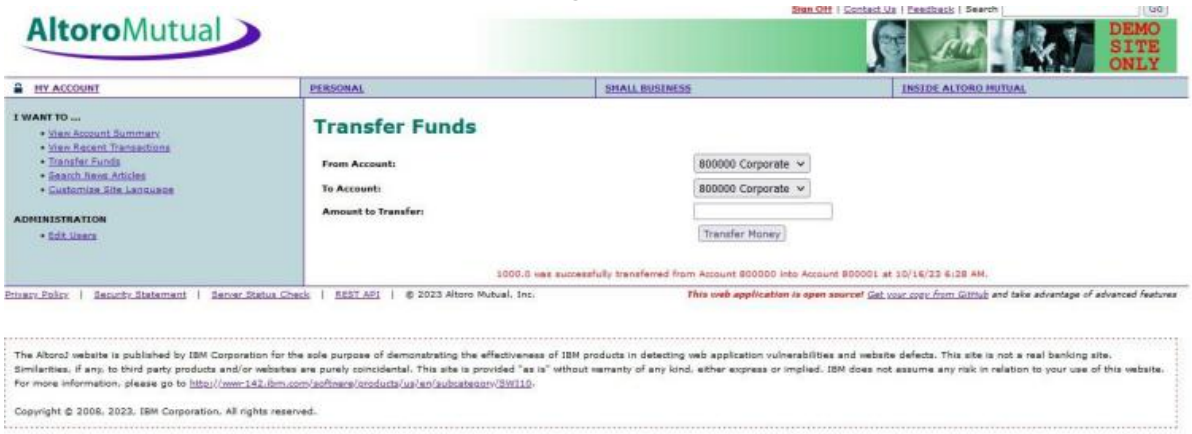
Open burp and notice the change.

In the 15th line change the amount from 100 to 1000 and click forward

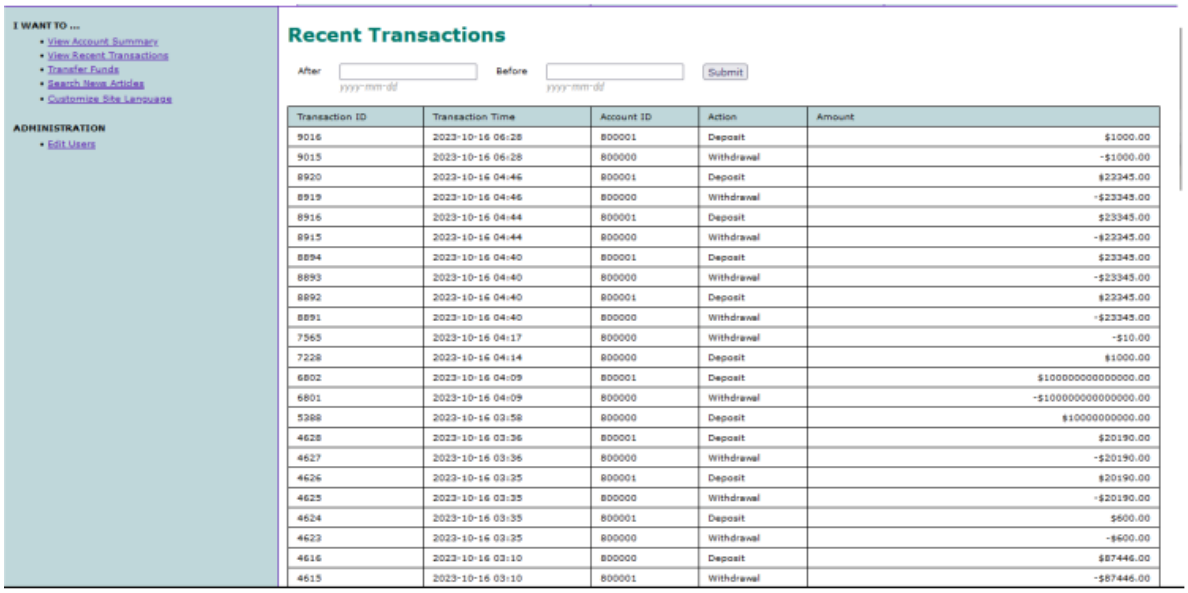
```
fromAccount=800000&toAccount=800001&transferAmount=100&transfer=Transfer+Money

fromAccount=800000&toAccount=800001&transferAmount=1000&transfer=Transfer+Money
```



Look at the site, we can notice the msg that shows the transfer of 1000.



➢ Off the Intercept and open “View Recent Transactions”



Transaction ID	Transaction Time	Account ID	Action	Amount
9016	2023-10-16 06:28	800001	Deposit	$1000.00
9015	2023-10-16 06:28	800000	Withdrawal	-$1000.00
8920	2023-10-16 04:46	800001	Deposit	$23345.00
8919	2023-10-16 04:46	800000	Withdrawal	-$23345.00
8916	2023-10-16 04:44	800001	Deposit	$23345.00
8915	2023-10-16 04:44	800000	Withdrawal	-$23345.00
8894	2023-10-16 04:40	800001	Deposit	$23345.00
8893	2023-10-16 04:40	800000	Withdrawal	-$23345.00
8892	2023-10-16 04:40	800001	Deposit	$23345.00
8891	2023-10-16 04:40	800000	Withdrawal	-$23345.00
7565	2023-10-16 04:17	800000	Withdrawal	-$10.00
7228	2023-10-16 04:14	800000	Deposit	$1000.00
6802	2023-10-16 04:09	800001	Deposit	$100000000000000.00
6801	2023-10-16 04:09	800000	Withdrawal	-$100000000000000.00
5289	2023-10-16 03:58	800000	Deposit	$10000000000.00
4628	2023-10-16 03:36	800001	Deposit	$20190.00
4627	2023-10-16 03:36	800000	Withdrawal	-$20190.00
4626	2023-10-16 03:35	800001	Deposit	$20190.00
4625	2023-10-16 03:35	800000	Withdrawal	-$20190.00
4624	2023-10-16 03:35	800001	Deposit	$600.00
4623	2023-10-16 03:35	800000	Withdrawal	-$600.00
4616	2023-10-16 03:10	800000	Deposit	$87446.00
4615	2023-10-16 03:10	800001	Withdrawal	-$87446.00

Recommendation:

• Utilize parameterized queries or prepared statements when interacting with databases. This prevents user-controlled input from being executed as SQL code.

• Implement a Content Security Policy to mitigate the risk of cross-site scripting (XSS) attacks, which can be used to execute unauthorized queries.

• Implement account lockout mechanisms that temporarily suspend accounts after a specified number of unauthorized access attempts to prevent brute force attacks.

6. Vulnerability name: Security Misconfiguration

CWE:732 Description: The product specifies permissions for a security-critical resource in a way that allows that resource to be read or modified by unintended actors.

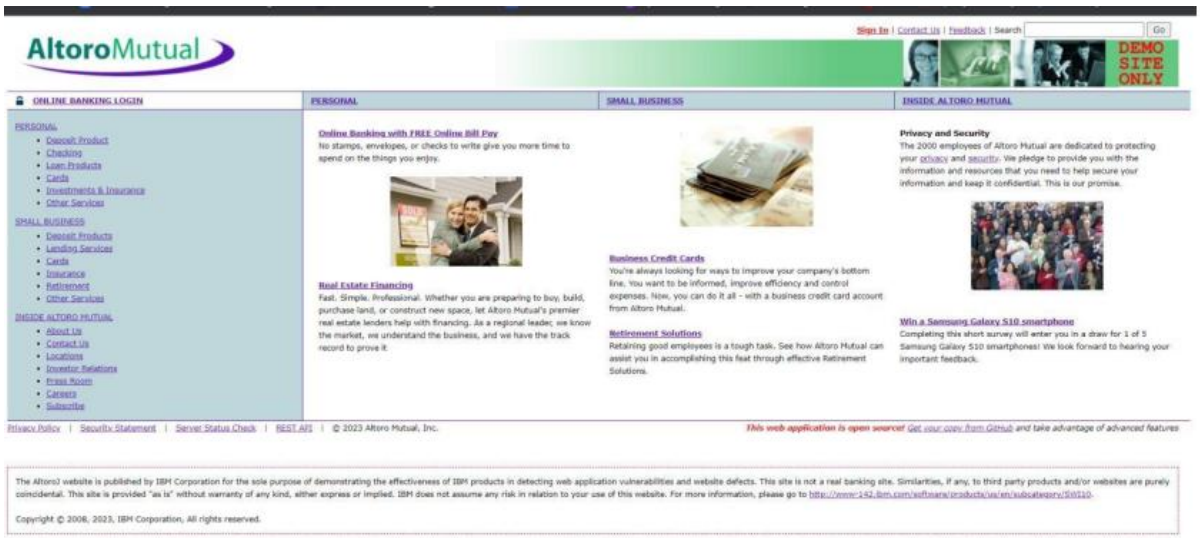
Business Impact: To effectively lessen the business consequences stemming from CWE-732, organizations must place a strong emphasis on bolstering their configuration management practices. This involves the establishment of secure default settings, the routine conduct of comprehensive security evaluations, and the deployment of intrusion detection systems to promptly detect and address misconfigurations and vulnerabilities. These actions are of paramount importance in safeguarding sensitive data, fortifying system security, and preserving the organization's esteemed reputation.

Vulnerability path: http://testfire.net/

Vulnerability parameter: http://testfire.net/login.jsp

Steps to Reproduce:

➢ Access the URL



➢ We will be using the uniscan tool which is available in the kali

```
┌──(kali㉿kali)-[~]
└─$ sudo uniscan -h
[sudo] password for kali:
#####################################
# Uniscan project                   #
# http://uniscan.sourceforge.net/   #
#####################################
V. 6.3

OPTIONS:
        -h      help
        -u      <url> example: https://www.example.com/
        -f      <file> list of url's
        -b      Uniscan go to background
        -q      Enable Directory checks
        -w      Enable File checks
        -e      Enable robots.txt and sitemap.xml check
        -d      Enable Dynamic checks
        -s      Enable Static checks
        -r      Enable Stress checks
        -i      <dork> Bing search
        -o      <dork> Google search
        -g      Web fingerprint
        -j      Server fingerprint

usage:
[1] perl ./uniscan.pl -u http://www.example.com/ -qweds
[2] perl ./uniscan.pl -f sites.txt -bqweds
[3] perl ./uniscan.pl -i uniscan
[4] perl ./uniscan.pl -i "ip:xxx.xxx.xxx.xxx"
[5] perl ./uniscan.pl -o "inurl:test"
[6] perl ./uniscan.pl -u https://www.example.com/ -r
```

➢ We will use this tool to scan for hidden directories that can be access due to poor security measures taken in this website.

```
┌──(kali㉿kali)-[~]
└─$ sudo uniscan -u http://testfire.net/ -qweds
[sudo] password for kali:
#####################################
# Uniscan project                   #
# http://uniscan.sourceforge.net/   #
#####################################
V. 6.3


Scan date: 17-10-2023 7:18:22
=============================================================================
| Domain: http://testfire.net/
| Server: Apache-Coyote/1.1
| IP: 65.61.137.117
=============================================================================
|
| Directory check:
| [+] CODE: 200 URL: http://testfire.net/admin/fckeditor/admins/
| [+] CODE: 200 URL: http://testfire.net/admin/
| [+] CODE: 200 URL: http://testfire.net/admin/scripts/fckeditor/
| [+] CODE: 200 URL: http://testfire.net/admin/FCKeditor/
| [+] CODE: 200 URL: http://testfire.net/aux/
| [+] CODE: 200 URL: http://testfire.net/bank/
=============================================================================
|
```

➢ We found directories that can be accessed easily. Also all the files that are present in the website has also been found.

```
File check:
[+] CODE: 200 URL: http://testfire.net/admin/account.asp
[+] CODE: 200 URL: http://testfire.net/admin/account.html
[+] CODE: 200 URL: http://testfire.net/admin/admin.php
[+] CODE: 200 URL: http://testfire.net/admin/admin_phpinfo.php4
[+] CODE: 200 URL: http://testfire.net/admin/account.php
[+] CODE: 200 URL: http://testfire.net/admin/admin.shtml
[+] CODE: 200 URL: http://testfire.net/admin/aindex.htm
[+] CODE: 200 URL: http://testfire.net/admin/auth.php
[+] CODE: 200 URL: http://testfire.net/admin/cfg/configscreen.inc.php+
[+] CODE: 200 URL: http://testfire.net/admin/cfg/configsite.inc.php+
[+] CODE: 200 URL: http://testfire.net/admin/cfg/configsql.inc.php+
[+] CODE: 200 URL: http://testfire.net/admin/cfg/configtache.inc.php+
[+] CODE: 200 URL: http://testfire.net/admin/cms/htmltags.php
[+] CODE: 200 URL: http://testfire.net/admin/config.php
[+] CODE: 200 URL: http://testfire.net/admin/contextAdmin/contextAdmin.html
[+] CODE: 200 URL: http://testfire.net/admin/controlpanel.asp
[+] CODE: 200 URL: http://testfire.net/admin/controlpanel.php
[+] CODE: 200 URL: http://testfire.net/admin/controlpanel.html
[+] CODE: 200 URL: http://testfire.net/admin/cplogfile.log
[+] CODE: 200 URL: http://testfire.net/admin/cp.php
[+] CODE: 200 URL: http://testfire.net/admin/cp.html
[+] CODE: 200 URL: http://testfire.net/admin/database/wwForum.mdb
[+] CODE: 200 URL: http://testfire.net/admin/credit_card_info.php
[+] CODE: 200 URL: http://testfire.net/admin/datasource.asp
[+] CODE: 200 URL: http://testfire.net/admin/db.php
[+] CODE: 200 URL: http://testfire.net/admin/home.asp
[+] CODE: 200 URL: http://testfire.net/admin/home.php
[+] CODE: 200 URL: http://testfire.net/admin/index.asp
[+] CODE: 200 URL: http://testfire.net/admin/index.html
[+] CODE: 200 URL: http://testfire.net/admin/index.php
[+] CODE: 200 URL: http://testfire.net/admin/phpinfo.php
[+] CODE: 200 URL: http://testfire.net/admin/settings.inc.php+
[+] CODE: 200 URL: http://testfire.net/admin/system_footer.php
[+] CODE: 200 URL: http://testfire.net/admin/upload.php
[+] CODE: 200 URL: http://testfire.net/admin/templates/header.php
[+] CODE: 200 URL: http://testfire.net/admin/wg_user-info.ml
[+] CODE: 200 URL: http://testfire.net/admin/script.php
[+] CODE: 200 URL: http://testfire.net/login.jsp
```

➢ Let us go through one webpage to see the vulnerability.

➢ Here we can see the sensitive data of job applications which can help an attacker.

Recommendations:

• Implement a change management process to document and review all changes to system configurations. This helps prevent unauthorized or unintended alterations.

• Follow security guidelines provided by software and hardware vendors to ensure secure configurations.

• Regularly perform compliance checks against industry standards and regulations to ensure that configurations remain in compliance with security requirements.

7.VulnerabilityName: Cross-Site Request Forgery

CWS: 352

OWASP Category: A08: Cross-Site Request Forgery

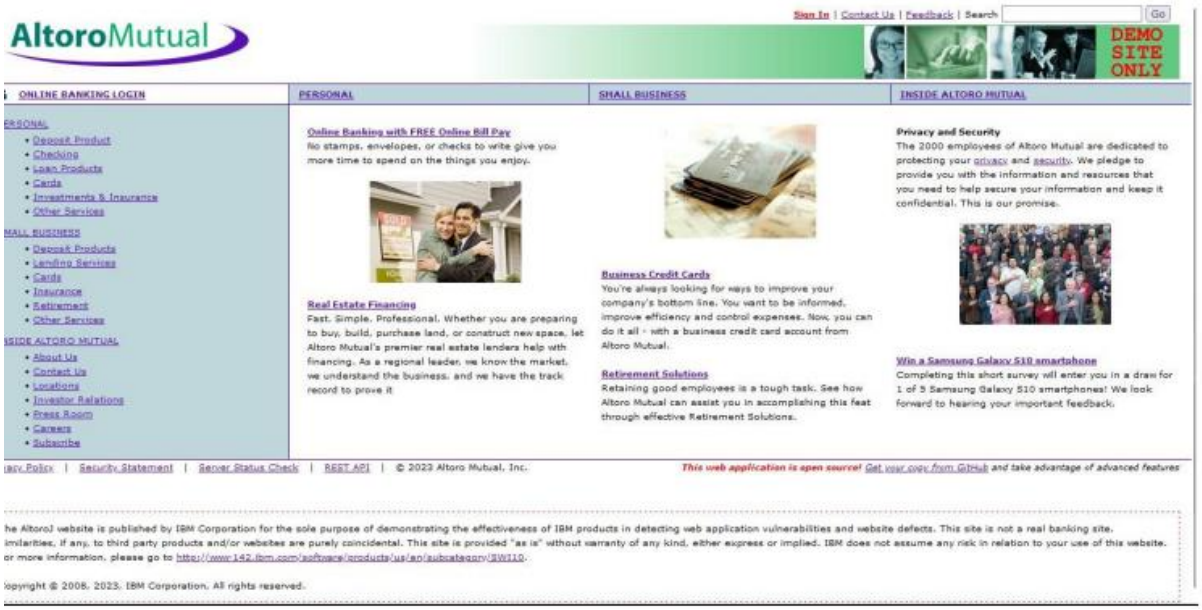
Description: Cross-Site Request Forgery (CSRF) is an attack that forces an end user to execute unwanted actions on a web application in which they're currently authenticated. With a little help of social engineering (such as sending a link via email or chat), an attacker may trick the users of a web application into executing actions of the attacker's choosing. If the victim is a normal user, a successful CSRF attack can force the user to perform state changing requests like transferring funds, changing their email address, and so forth. If the victim is an administrative account, CSRF can compromise the entire web application. Business Impact: To effectively reduce the business repercussions of CWE-352, organizations must prioritize secure coding practices, including the incorporation of anti-CSRF tokens, while also conducting routine security assessments and testing to detect and rectify CSRF vulnerabilities. Furthermore, educating users on secure browsing practices is pivotal in preventing CSRF attacks. These actions are of paramount importance in the protection of data and in upholding the trust of customers and partners.

Vulnerability Path: http://testfire.net/

Vulnerability Parameter: http://testfire.net/bank/transfer.jsp

Steps to Reproduce:

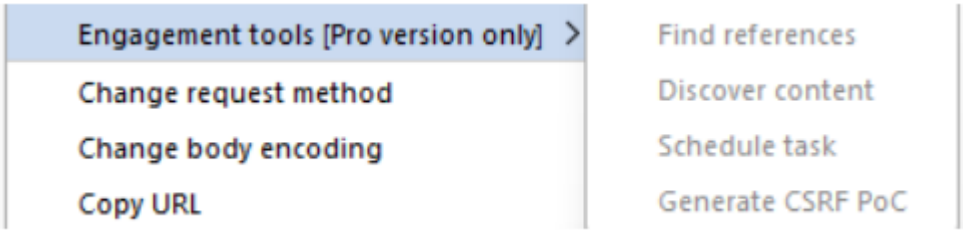
➢ Open https://testfire.net/.



➢ Login to it using default credentials, and you will find two accounts.

➢ Capture the request in any Intercepting proxy (Burp Suite, Charles Proxy etc.) while transferring the amount from one account to another as shown in the below picture.

➢ Generate CSRF POC using Burp Suite Engagement tools (by modifying the account numbers accordingly)



As the victim is already logged-in to the site an attacker can send the crafted malicious form/URL to the user to his email/text/by any other means and trick the victim using his social engineering skills/techniques like below.

```
Limited Time Offer Get a $100 Gift Card Now!!  (Evil :P)
```

➢ User thinks, he could use a free $100 gift card! What could go wrong? You open the email and click the link/open the attachment.

➢ Once the user opens the form a transaction is automatically performed from his account without his knowledge as the user is already logged in.

Recent Transactions

After yyyy-mm-dd Before yyyy-mm-dd Submit

Transaction ID	Transaction Time	Account ID	Action	Amount
12539	2023-10-17 06:38	800002	Deposit	$20000.00
12538	2023-10-17 06:38	800003	Withdrawal	-$20000.00
12537	2023-10-17 06:36	800003	Deposit	$1234.00
12536	2023-10-17 06:36	800003	Withdrawal	-$1234.00
12535	2023-10-17 06:36	800003	Deposit	$1234.00
12534	2023-10-17 06:36	800003	Withdrawal	-$1234.00
12533	2023-10-17 06:36	800003	Deposit	$1234.00
12532	2023-10-17 06:36	800003	Withdrawal	-$1234.00

Recommendations:

• Implement Synchronizer Token Patterns to include unique, randomly generated tokens in each HTTP request and validate these tokens on the server side to verify the authenticity of the request.

• Utilize the SameSite attribute in cookies to restrict their usage to same-site requests only, preventing them from being sent along with cross-site requests and effectively mitigating CSRF attacks.

• Utilize framework-specific protections and security features to prevent CSRF attacks. Many modern web frameworks have built-in mechanisms and libraries for handling CSRF vulnerabilities effectively.

• Implement a Content Security Policy to restrict the sources from which various types of content can be loaded. By specifying the trusted sources of content, you can minimize the risk of malicious code execution and reduce the likelihood of successful CSRF attacks.

7.Vulnerability name: Cleartext Transmission of Sensitive Information CWE: 319

Description:

The product transmits sensitive or security-critical data in cleartext in a communication channel that can be sniffed by unauthorized actors. Many communication channels can be "sniffed" (monitored) by adversaries during data transmission. For example, in networking, packets can traverse many intermediary nodes from the source to the destination, whether across the internet, an internal network, the cloud, etc. Some actors might have privileged access to a network interface or any link along the channel, such as a router, but

they might not be authorized to collect the underlying data. As a result, network traffic could be sniffed by adversaries, spilling security-critical data.

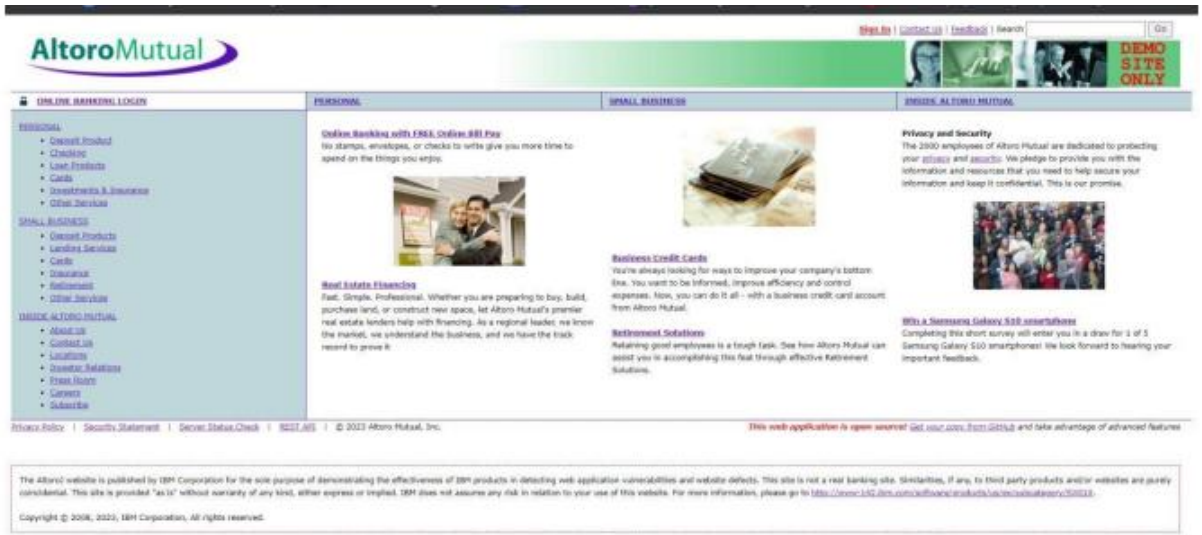
Business Impact: To effectively reduce the business consequences associated with CWE-319, organizations must prioritize the adoption of secure data transmission practices. This includes the utilization of encryption and robust, secure protocols. The routine conduct of security assessments and testing is pivotal in pinpointing and remedying vulnerabilities linked to data transmission. Furthermore, the education of users on secure data handling practices plays a vital role in proactively preventing data exposure incidents. These actions are of paramount importance in the protection of sensitive data and in preserving the trust of both customers and partners.

Vulnerability path: http://testfire.net/

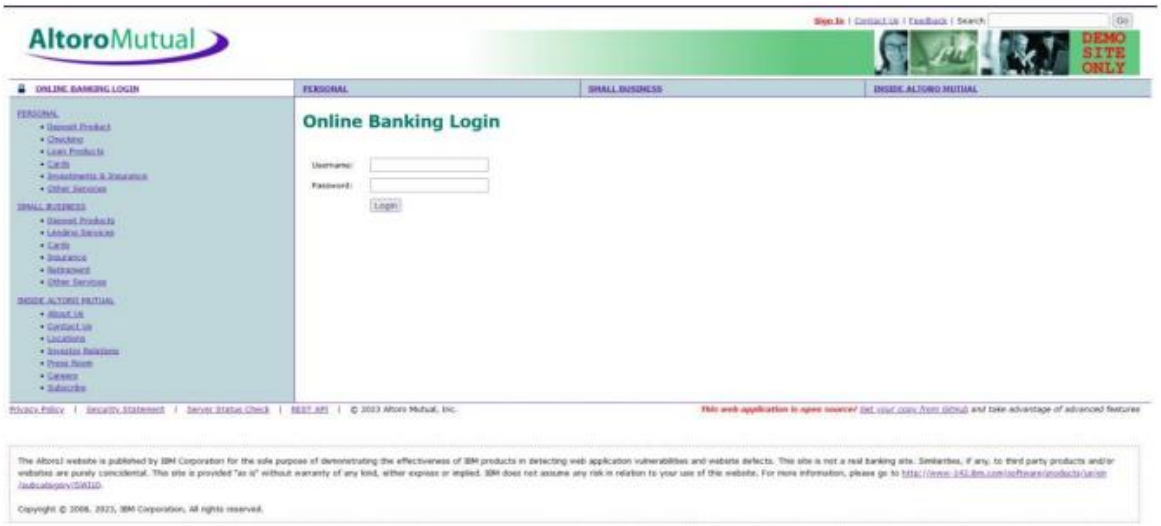
Vulnerability parameter: http://testfire.net/login.jsp

Steps to reproduce:

➢ Access the URL

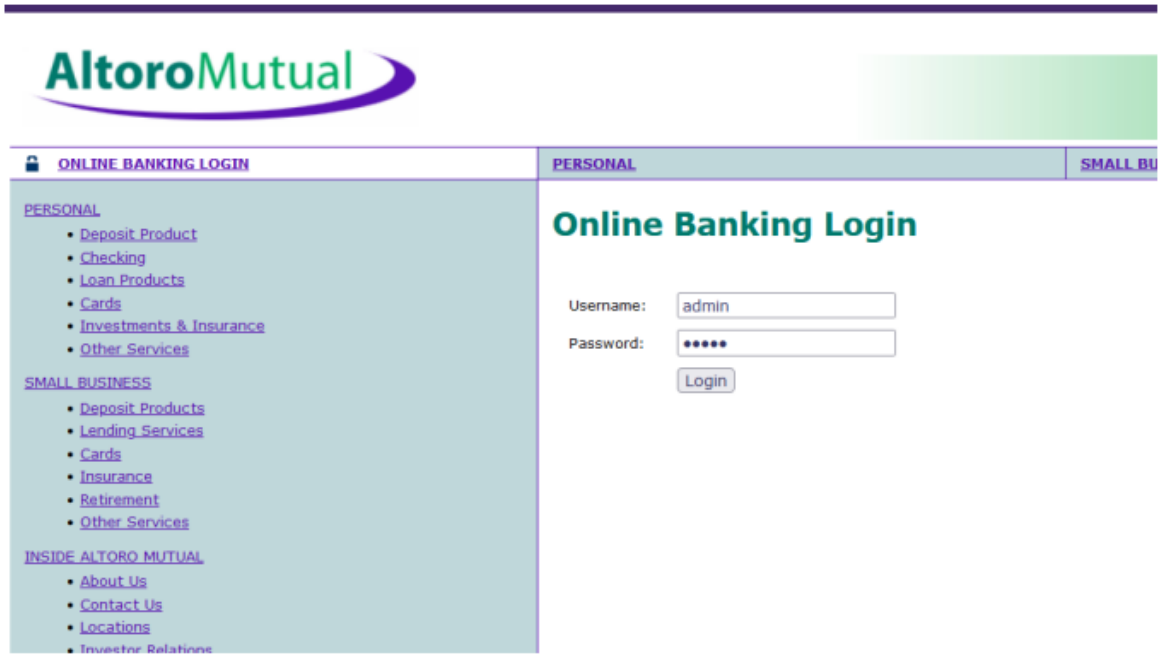


➢ Now we will try to sign in to this website with admin privileges



➢ Also, we will be using burp suite to get requests from the website and know additional information.

➢ We use ‘admin’ for the username and password.



➢ This request has been received in the burp suite with the username and password as well in clear text.

```
Pretty   Raw   Hex
1  POST /doLogin HTTP/1.1
2  Host: testfire.net
3  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:109.0) Gecko/20100101 Firefox/117.0
4  Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
5  Accept-Language: en-US,en;q=0.5
6  Accept-Encoding: gzip, deflate, br
7  Content-Type: application/x-www-form-urlencoded
8  Content-Length: 37
9  Origin: http://testfire.net
10 Connection: close
11 Referer: http://testfire.net/login.jsp
12 Cookie: JSESSIONID=B6ADB36ACD5CB3083787343A1F97F863; AltoroAccounts="ODAwMDAwfkNvcnBvcmF0ZX45LjQ30TAlMTE2MUU3fDgwMDAwMDX5DaGVjaZluZ34tNC4yMjc0NjkONTZFN3w="
13 Upgrade-Insecure-Requests: 1
14
15 uid=admin&passw=admin&btnSubmit=Login
```

➢ Now we click on forward request in the burp suite and then we will be redirected to the admin user details. Here in the burp suite, we can clearly see the login details in clear text. this is the clear indication of the vulnerability which can lead to data breach, monitored, and manipulated as well.

Recommendations:

• Educate users and employees on secure data handling practices, such as recognizing secure websites (look for "https") and avoiding insecure Wi-Fi networks.

• Employ data masking or redaction techniques to replace sensitive data with placeholders or cryptographic representations during transmission. • Implement encryption for data in transit. Use secure encryption protocols such as TLS/SSL to protect sensitive information during transmission.

• Use secure communication protocols for transmitting data, such as HTTPS for web applications, and ensure that the selected protocols are kept up-to-date.

9. Vulnerability name: Click Jacking

CWE: 1021

Description: The web application does not restrict or incorrectly restricts frame objects or UI layers that belong to another application or domain, which can lead to user confusion about which interface the user is interacting with.

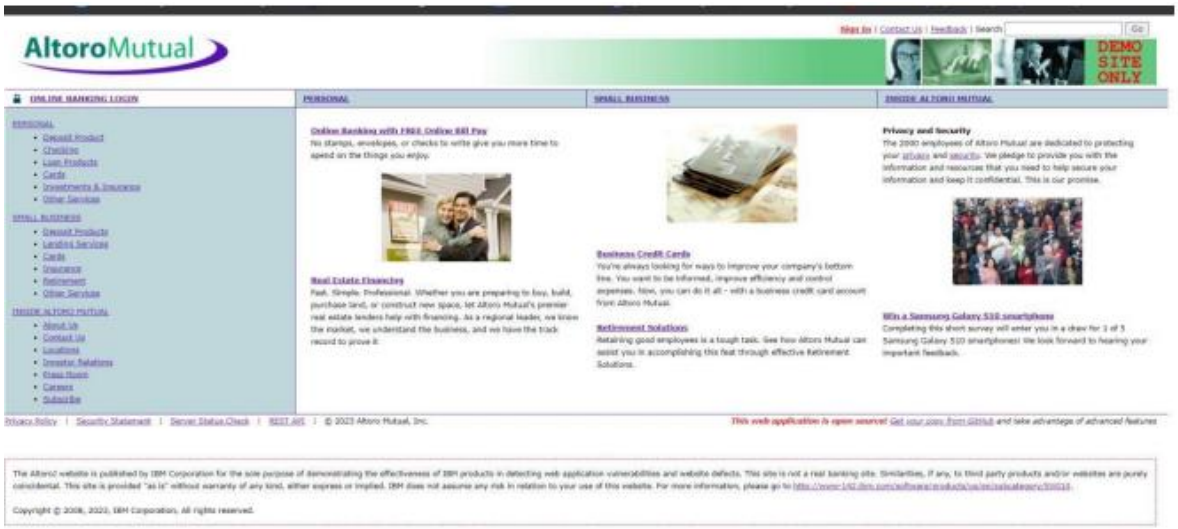
Business Impact: To effectively reduce the business consequences resulting from CWE-1021, organizations must prioritize the implementation of protective measures, such as frame-busting code. Simultaneously, educating users on safe browsing practices plays a crucial role in preventing clickjacking incidents. Additionally, routine security assessments and testing are pivotal for identifying and mitigating vulnerabilities associated with clickjacking. These actions are of paramount importance in upholding user trust, ensuring data protection, and safeguarding the organization's reputation.

Vulnerability path: http://testfire.net/

Vulnerability parameter: http://testfire.net/

Steps to reproduce:

➢ Access the URL



➢ Then take the URL and use it for the html code ➢ we will be writing some html code to perform this vulnerability. we will be writing the code in vs code for better flexibility and functionality.

```
<html>
<body>
<title>Click jacking vulnerability</title>
<h2>This website is vulnerable to clickjacking</h2>
<iframe  src="http://testfire.net/"></iframe>
</body>
</html>
```

➢ After the code has been written then we will be executing the code in the browser.

This website is vulnerable to clickjacking

➢ From this image we can see that the vulnerability has been found.
Recommendation:

• Set the X-Frame-Options HTTP response header to deny or same-origin to control how your site can be framed. This is supported by most modern browsers

. • Utilize frame-busting JavaScript code in web applications to prevent the embedding of your site within malicious iframes. This code can disrupt clickjacking attempts.

• Implement a Content Security Policy to restrict which domains can embed your site in iframes. This can help prevent unauthorized framing. • Implement additional security controls to preventUI redress attacks, such as clickjacking, within your web application.

10. Vulnerability name: web server allows password auto-completion CWE: 310

Description:

Weaknesses in this category are related to the design and implementation of data confidentiality and integrity. Frequently these deal with the use of encoding techniques, encryption libraries, and hashing algorithms. The weaknesses in this category could lead to a degradation of the quality data if they are not addressed.

Business impact: To effectively lessen the business consequences associated with CWE-310, organizations must prioritize the adoption of secure cryptographic practices. This entails ensuring proper password storage and

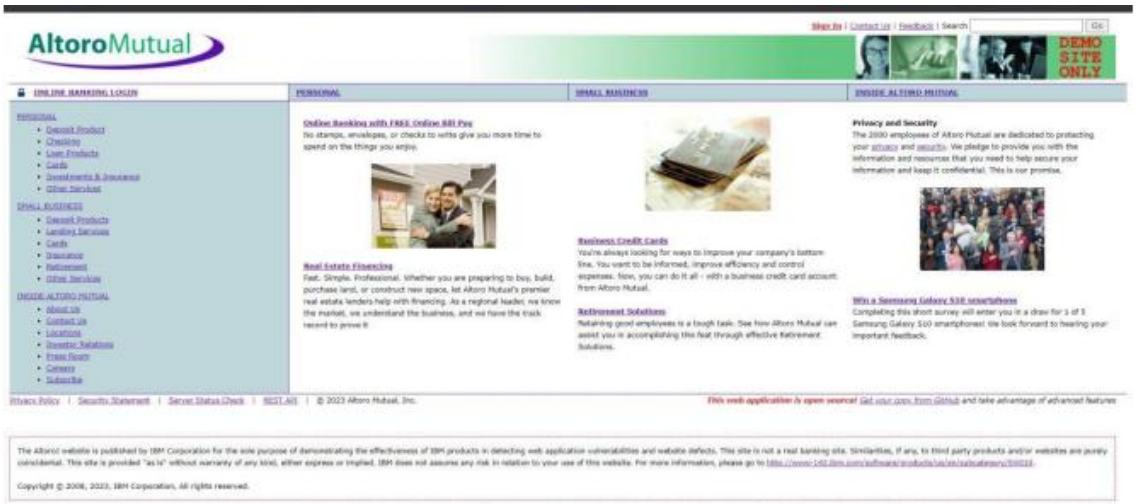
robust encryption key management. Simultaneously, conducting routine security assessments and testing is pivotal in detecting and mitigating vulnerabilities related to cryptographic issues. Striving for compliance with pertinent data protection regulations and industry standards is equally essential. These actions are of paramount importance in the protection of sensitive data, the upholding of user trust, and the preservation of the organization's esteemed reputation.

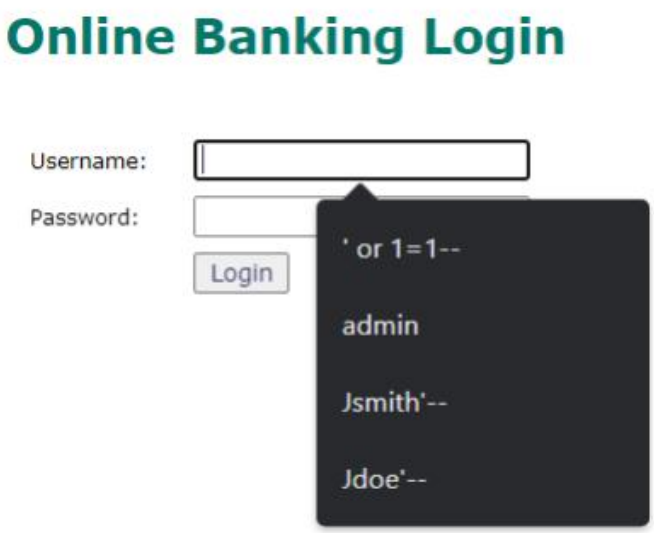
vulnerability path: http://testfire.net/

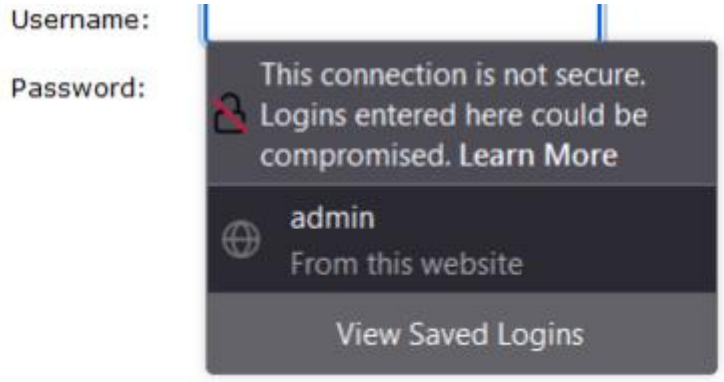
Vulnerability parameter: http://testfire.net/login.jsp

Steps to reproduce:

➢ Access the URL







From this image we can see the usernames and the passwords getting auto filled. This is a potential vulnerability as this can be a doorway for attackers. Responsibilities:

• Implement secure password storage mechanisms, such as using strong and salted cryptographic hashing algorithms like bcrypt or scrypt.

• Implement robust encryption key management practices, including secure key storage, key rotation, and access controls.

• Utilize well-established and proven cryptographic libraries and algorithms for encryption and decryption.

• Use secure encryption protocols, like TLS/SSL, for data transmission over networks to protect data in transit.

Stage – 2

Overview

A Nessus scan overview generally includes several important stages involved in identifying vulnerabilities in systems and networks.

1. Preparation:
First, the objectives of the scan are determined. This includes defining the scope of the scan, identifying the systems or networks to be evaluated, and deciding which types of vulnerabilities should be checked. It is also necessary to ensure that proper permissions and access rights are available to scan the target systems.

2. Configuration:
In this step, a scan policy is created to specify how the scan should be conducted. The policy includes details such as the types of vulnerabilities to detect, scanning schedules, and configuration settings. After that, the target assets are defined by entering IP addresses, hostnames, or network ranges of the systems to be scanned.

3. Scanning:
Once the configuration is completed, the Nessus scan is initiated using the selected policy and target assets. Nessus then systematically analyzes the systems to detect vulnerabilities using its database of plugins and predefined rules.

4. Vulnerability Identification:
During the scanning process, Nessus performs a thorough evaluation of the systems to identify weaknesses such as misconfigurations, open ports, and security vulnerabilities. The tool classifies the discovered vulnerabilities according to their severity levels, helping security teams prioritize which issues should be addressed first.

5. Reporting:
After the scan is finished, Nessus generates detailed reports describing the vulnerabilities that were found. These reports usually include the severity level of each issue, suggested remediation steps, and supporting technical details. Reports can also be customized and exported in formats like PDF, HTML, or CSV.

6. Remediation:
Security and IT teams examine the scan results to determine which vulnerabilities require immediate attention. Corrective actions are then taken to resolve the issues. This may include installing patches, modifying system configurations, or implementing additional security controls.

7. Follow-up:
Organizations often conduct Nessus scans regularly to continuously monitor the security status of their systems and networks. This helps detect newly emerging vulnerabilities and evaluate the progress of previously implemented fixes. Nessus can also be integrated with other security tools to automate vulnerability management processes.

8. Compliance Auditing:
If compliance checks are included in the scan policy, the results are reviewed to ensure that systems comply with required security standards and regulatory guidelines.

9. Documentation:
All scan results, remediation activities, and compliance records should be properly documented for auditing, monitoring, and reporting purposes.

10. User Training:
It is essential that IT and security personnel are properly trained to use Nessus effectively and accurately interpret scan results.

Nessus scans play an important role in proactive cybersecurity practices. They help organizations detect vulnerabilities in their IT infrastructure and reduce the risk of cyberattacks and data breaches. The frequency and depth of these scans depend on an organization's security policies and operational requirements.

About Nessus

Nessus is a widely used vulnerability scanning tool designed to identify security weaknesses in computer systems and networks. Security professionals and organizations use it to evaluate system security and proactively detect vulnerabilities that attackers might exploit.

1. Vulnerability Assessment:
Nessus performs vulnerability assessments by scanning systems or networks to

detect security flaws such as missing patches, configuration errors, weak passwords, and other potential threats.

2. Network Scanning:
The tool can analyze many types of network devices including servers, routers, switches, and firewalls. It checks open ports, network services, and installed software to identify possible vulnerabilities.

3. Plugin Architecture:
Nessus operates using a plugin-based system. These plugins enable the scanner to detect a wide range of vulnerabilities and are regularly updated to include checks for newly discovered security threats.

4. Compliance Auditing:
Nessus can also evaluate systems against security standards and compliance requirements such as PCI DSS or CIS benchmarks, helping organizations maintain regulatory compliance.

5. Customization:
Users can customize Nessus scans according to their specific needs by defining scan policies, selecting target systems, and choosing the types of vulnerabilities to examine.

6. Reporting:
The tool provides detailed reports that list detected vulnerabilities, their severity levels, and recommended solutions. These reports help IT teams prioritize and address security issues efficiently.

7. Scanning Frequency:
Organizations typically conduct Nessus scans regularly to maintain strong security and stay protected from newly emerging threats.

8. Integration:
Nessus can integrate with other cybersecurity tools and platforms, allowing automated scanning and more efficient vulnerability management.

9. Licensing:
Nessus is available in both free and commercial versions. The free edition offers limited features, while the paid version provides advanced capabilities and support.

10. User-Friendly Interface:
Nessus includes a web-based interface that is easy to use, making it accessible not only for cybersecurity professionals but also for general IT users.

It is important to understand that while Nessus identifies vulnerabilities, it does not automatically fix them. The responsibility of resolving these issues lies with the organization's IT and security teams.

s.no	Vulnerability Name	Severity	Plugins
1	Cross-site scripting (stored)	Low	Web application Scanner
2	Broken Access Control	Medium	SSH weak HMAC Algorithms Supported
3	SQL Injection	Medium	N/A
4	Broken Authentication	Low	HSTS Missing from server
5	Insecure Direct Object Reference	Info	Service Detection
6	Security Misconfiguration	Low	SSL cipher Block Chaining cipher suites
7	Cross-Site Request Forgery	Low	HTTP Form- Based Authentication Detected
8	Cleartext Transmission of Sensitive Information	Info	Cleartext Transmission ofSensitive Information over HTTP
9	Click Jacking	Medium	HTML Rendering Layer Vulnerability
10	webserver allows password autocompletion	low	SSL/TLS Diffie-Hellman Modulus<=1024 bits(Logjam)

Target Website: testfire.net
Target IP Address: 54.83.41.36

---

1. Vulnerability Name: Cross-Site Scripting (Stored)

Severity: Low
Plugin: Web Application Scanner
Port: N/A

Description:
Stored Cross-Site Scripting (XSS) occurs when malicious scripts are permanently stored in a vulnerable web application. When other users access the affected page, the malicious script is executed in their browsers. Reflected XSS occurs when a web application returns user input directly to the browser without proper validation or sanitization.

Solution:

- Use parameterized or prepared database statements to reduce the risk of SQL injection and XSS attacks.
- Validate and sanitize all user inputs and ensure output data is properly encoded before displaying it on web pages.

Business Impact:
Malicious content may be stored in trusted locations such as databases, comment sections, or visitor logs. When this data is later displayed to users, the harmful script can execute in their browsers. Attackers may target users with higher privileges to gain unauthorized access, perform actions on their behalf, or steal sensitive data.

---

2. Vulnerability Name: Broken Access Control

Severity: Medium
Plugin: SSH Weak HMAC Algorithms Supported
Port: N/A

Description:
Broken access control occurs when authorization checks are missing or inconsistently applied. This allows users to access restricted data or perform actions they should not be permitted to do.

Solution:

- Implement strong authentication and authorization mechanisms.
- Verify user permissions before allowing sensitive operations.
- Use secure session management practices such as strong session IDs, session expiration, and session token regeneration.

Business Impact:
This vulnerability can lead to financial loss, damage to organizational reputation, legal issues, operational disruptions, and loss of customer confidence.

---

3. Vulnerability Name: SQL Injection

Severity: Medium
Plugin: N/A
Port: 80/443

Description:
SQL Injection occurs when an application constructs SQL queries using external input without properly sanitizing it. Attackers can manipulate the input to modify the SQL query and access or manipulate the database.

Solution:

- Use parameterized queries or prepared statements when accessing databases.
- Apply strict input validation and sanitization methods.
- Use stored procedures to separate SQL logic from application code.

Business Impact:
SQL Injection can lead to serious consequences including data breaches, financial losses, reputational damage, legal liabilities, and operational

disruptions. Preventing and fixing these vulnerabilities is essential to maintain application security.

---

4. Vulnerability Name: Broken Authentication

Severity: Low
Plugin: HSTS Missing from HTTPS Server (Plugin ID: 142960)
Port: 443

Description:
This vulnerability occurs when the system fails to properly restrict user access, allowing one user to access another user's data by manipulating identifying parameters.

Solution:

- Implement strong authentication mechanisms such as Multi-Factor Authentication (MFA).
- Apply role-based access control (RBAC) to manage user permissions.
- Use intrusion detection and prevention systems to monitor suspicious authentication activities.

Business Impact:
Organizations should strengthen authentication and session management processes, secure credential storage, and conduct regular security assessments to protect sensitive information and maintain system security.

---

5. Vulnerability Name: Insecure Direct Object Reference (IDOR)

Severity: Informational
Plugin: Service Detection (Plugin ID: 22964)
Port: 80/443

Description:
IDOR occurs when attackers can manipulate object identifiers in URLs or request parameters to access unauthorized data due to missing access control checks.

Solution:

- Use prepared statements when interacting with databases.
- Implement Content Security Policies to reduce XSS risks.
- Apply account lockout policies after multiple failed login attempts.

Business Impact:
Strengthening authorization mechanisms and performing regular security testing can prevent unauthorized data access and protect sensitive information and organizational reputation.

---

6. Vulnerability Name: Security Misconfiguration

Severity: Low
Plugin: SSL Cipher Block Chaining Cipher Suites
Port: 443

Description:
Security misconfiguration occurs when improper permissions or insecure settings allow unauthorized users to access or modify critical resources.

Solution:

- Establish a change management process for configuration updates.
- Follow vendor security guidelines for secure configuration.
- Perform regular compliance checks against security standards.

Business Impact:
Proper configuration management helps prevent vulnerabilities, protect sensitive information, and maintain strong system security.

---

7. Vulnerability Name: Cross-Site Request Forgery (CSRF)

Severity: Low
Plugin: HTTP Form-Based Authentication Detected
Port: 8080/443

Description:
CSRF attacks force authenticated users to perform unwanted actions on a web

application. Attackers may trick users into clicking malicious links that execute unauthorized requests.

Solution:

- Implement CSRF tokens in HTTP requests and validate them on the server side.
- Use the SameSite cookie attribute to prevent cross-site requests.
- Apply framework security features designed to prevent CSRF attacks.
- Implement a Content Security Policy to limit malicious content.

Business Impact:
Using secure coding practices, performing regular testing, and educating users about secure browsing can help prevent CSRF attacks and protect sensitive data.

---

8. Vulnerability Name: Cleartext Transmission of Sensitive Information

Severity: Medium
Plugin: Cleartext Transmission of Sensitive Information over HTTP
Port: 443

Description:
Sensitive data is transmitted without encryption, making it vulnerable to interception by attackers who monitor network traffic.

Solution:

- Use encryption protocols such as TLS/SSL to secure data during transmission.
- Use secure communication methods such as HTTPS.
- Apply data masking or redaction techniques for sensitive information.
- Educate users about secure network usage.

Business Impact:
Implementing strong encryption and secure protocols protects sensitive data and maintains customer trust.

---

9. Vulnerability Name: Clickjacking

Severity: Informational
Plugin: HTML Rendering Layer Vulnerability
Port: 80/443

Description:
Clickjacking occurs when attackers trick users into interacting with hidden or malicious UI elements by embedding a webpage inside another frame.

Solution:

- Configure the **X-Frame-Options** header to prevent unauthorized framing.
- Implement frame-busting scripts to block embedding attempts.
- Use Content Security Policy to restrict iframe sources.

Business Impact:
Implementing these protections helps maintain user trust and prevents attackers from manipulating user actions.

---

10. Vulnerability Name: Web Server Allows Password Auto-Completion

Severity: Low
Plugin: SSL/TLS Diffie-Hellman Modulus ≤ 1024 Bits (Logjam)
Port: 443

Description:
This vulnerability relates to weak cryptographic implementations and insecure password handling mechanisms that may compromise data confidentiality and integrity.

Solution:

- Use strong cryptographic hashing algorithms with salting for password storage.
- Implement secure encryption key management.
- Use trusted cryptographic libraries.
- Apply secure protocols such as TLS/SSL for data transmission.

Business Impact:
Organizations should adopt strong cryptographic practices, perform regular security assessments, and comply with data protection regulations to safeguard sensitive information and maintain trust.

s.no	VULNERABILITY NAME	SEVERITY	PLUGINS
1	Cross-site scripting (DOM based XSS)	Medium	Multiple XSS
2	Information exposure (web server HTTP header internal IP disclosure)	Medium	CGI Abuse: XSS
3	Information exposure (SSL (Multiple Issues))	Low	Webserver HTTP Header Internal IP Disclosure
4	Information exposure (HTTP (Multiple Issues))	Info	SSL Certificate Information
5	Information exposure (HTTP (Multiple Issues))	Info	HTTP Information
6	OS identification failed	Info	TCP/IP Remote Probing

Target Website: vit.ac.in
Target IP Address: 122.184.65.22

---

1. Vulnerability Name: Cross-Site Scripting (DOM-Based XSS)

Severity: Medium
Plugin: Multiple XSS
Port: 443

Description:
Cross-Site Scripting (XSS) occurs when a harmful script is inserted into a vulnerable web application. In reflected XSS, the malicious script is returned by the web application and executed in the user's browser. DOM-based XSS occurs when the vulnerability exists within the Document Object Model (DOM). The DOM is a programming interface for HTML and XML documents that allows programs to modify the structure, content, and style of a webpage.

Solution:

- Use parameterized queries or prepared statements when accessing databases to prevent SQL injection and reduce the risk of XSS attacks.
- Validate and sanitize all user inputs and ensure proper output encoding before displaying data on web pages.
- Avoid unsafe DOM manipulation methods such as innerHTML and document.write, which may allow scripts to execute.
- Use trusted JavaScript frameworks that include built-in security features to prevent DOM-based XSS vulnerabilities.

Business Impact:
If harmful data is stored in areas such as databases, comment sections, or visitor logs, it may later be displayed as part of dynamic content. Attackers usually target sections viewed by many users or users with high privileges. If privileged users execute malicious scripts, attackers may gain unauthorized access to sensitive information or perform actions on behalf of those users.

---

2. Vulnerability Name: jQuery 1.2 < 3.5.0 Multiple XSS

Severity: Medium
Plugin: CGI Abuse: XSS
Port: 443

Description:
This vulnerability occurs when malicious scripts are injected into a vulnerable web application using outdated versions of the jQuery library. Stored XSS occurs when harmful scripts are permanently saved on the server or database, often in user-generated content. Reflected XSS occurs when malicious scripts are included in URLs and executed when a user clicks a manipulated link. DOM-based XSS occurs within the DOM structure of the webpage, allowing attackers to manipulate webpage content and execute scripts.

Solution:

- Upgrade to the latest version of jQuery to apply security patches and improvements.

- Monitor security announcements related to jQuery and update libraries regularly.
- Implement a Content Security Policy (CSP) to limit the sources from which scripts can be loaded.
- Review application code and ensure that user inputs are properly sanitized and validated to prevent XSS vulnerabilities.

Business Impact:
Malicious scripts stored in trusted locations such as databases or message boards can be executed whenever users view the affected content. If high-privileged users interact with the infected content, attackers may perform unauthorized actions or access sensitive information.

---

3. Vulnerability Name: Information Exposure (Web Server HTTP Header Internal IP Disclosure)

Severity: Low
Plugin: Web Server HTTP Header Internal IP Disclosure
Port: 80

Description:
This vulnerability occurs when sensitive information, such as internal IP addresses, is unintentionally revealed through HTTP response headers. This information can be used by attackers to understand the internal network structure.

Solution:

- Review web server configuration files such as **httpd.conf** or **nginx.conf** to ensure internal IP addresses are not included in response headers.
- Remove or replace internal IP addresses with appropriate public identifiers.
- Configure proxy servers or load balancers to hide internal network details.
- Modify HTTP headers like **Server** or **X-Powered-By** to avoid revealing unnecessary system information.

Business Impact:
To reduce the risks associated with such exposure, organizations must maintain secure SSL/TLS configurations and conduct regular security testing. Following data protection standards helps protect sensitive information and maintain user trust.

---

4. Vulnerability Name: Information Exposure (SSL Multiple Issues)

Severity: Informational
Plugin: SSL Certificate Information
Port: 443

Description:
This issue arises when SSL configuration exposes sensitive details to unauthorized users. Improper SSL configurations may reveal system information that attackers could exploit.

Solution:

- Use modern encryption protocols such as TLS 1.2 or higher and disable outdated protocols like SSLv3.
- Implement Perfect Forward Secrecy to protect previously encrypted communications.
- Store and manage private encryption keys securely to prevent unauthorized access.

Business Impact:
Implementing strong SSL/TLS security configurations and conducting regular assessments are essential to protect sensitive data, comply with regulations, and maintain organizational credibility.

---

5. Vulnerability Name: Information Exposure (HTTP Multiple Issues)

Severity: Informational
Plugin: HTTP Information
Port: 80 / 443

Description:
This vulnerability occurs when a web server unintentionally reveals sensitive information through HTTP communication or server responses.

Solution:

- Reduce unnecessary redirects and use **301 permanent redirects** for stable content navigation.
- Identify and fix broken links to improve website reliability.
- Configure **CORS** settings properly to control which domains can access the server.
- Upgrade the website from **HTTP to HTTPS** to ensure secure data transmission.

Business Impact:
Regular monitoring and maintenance of web services are essential to maintain security and performance. Addressing HTTP issues quickly helps maintain user trust and ensures smooth online operations.

---

6. Vulnerability Name: OS Identification Failed

Severity: Informational
Plugin: TCP/IP Remote Probing
Port: N/A

Description:
OS Identification Failed indicates that the scanning system could not accurately detect the operating system running on the target device. This may occur due to configuration issues, network restrictions, or compatibility problems. Failure to identify the operating system may affect vulnerability detection and security management.

Solution:

- Avoid excessive redirects and maintain proper website structure.
- Regularly check and repair broken links on the website.
- Configure **CORS** policies appropriately to control server access.
- Implement **HTTPS** to improve data security and user trust.

Business Impact:
Failure to correctly identify the operating system may result in operational challenges, potential security risks, compliance issues, and reduced customer trust. Addressing the problem promptly helps prevent downtime, protects sensitive data, and maintains the organization's reputation.

Stage – 3

Title: Enforce least-privilege access through identity-aware, application-specific access policies using Zscaler Private Access (ZPA).

Zero Trust Network Access (ZTNA)

Zero Trust Network Access (ZTNA) is a modern cybersecurity approach that changes traditional access control from a network-focused model to an identity-focused model. The key idea behind ZTNA is **"never trust, always verify."** This means that no user or device is automatically trusted, whether they are inside or outside the organization's network. Unlike traditional remote access systems that provide users with wide access to the entire network, ZTNA creates a secure connection only to the specific application the user is allowed to use. As a result, unauthorized users cannot see or reach the application, which reduces the attack surface and prevents attackers from moving laterally across the network.

Zscaler Private Access (ZPA)

Zscaler Private Access (ZPA) is a cloud-based ZTNA solution that enables secure access to private applications based on user identity and context, without providing direct network access. In this model, users first connect to the Zscaler cloud, which then establishes a secure connection to the required application through **ZPA App Connectors** located in the data center or cloud. Since ZPA does not place users directly on the corporate network, internal IP addresses and network infrastructure remain hidden. Access policies are applied dynamically using factors such as user identity, group membership, device status, and location, ensuring that users can access only the applications they are authorized to use.

Principle of Least Privilege

The Principle of Least Privilege (PoLP) is a security practice that ensures a user, device, or process receives only the minimum permissions needed to perform its task. In network security, this approach replaces broad network access with more limited, application-level access. When least privilege is implemented through ZPA, even if an attacker gains access to a user's credentials, they cannot move to other systems because the rest of the network remains hidden. This method helps reduce insider threats and limits the impact of security breaches.

Identity-Aware Access

Identity-aware access is a security approach where access decisions are based on the user's identity rather than their network location. ZPA integrates with identity providers such as **Microsoft Active Directory, Azure AD, Okta, and Ping Identity**. This integration allows organizations to use existing user roles, attributes, and group memberships to create detailed access policies. For instance, a rule may allow only employees in the Finance department who are using company-managed devices to access the SAP Finance application during working hours. This ensures that access is closely linked to both the user's identity and their context.

Application-Specific Access Policies

Application-specific access policies are the primary enforcement mechanism in ZPA. Instead of allowing traffic to entire network segments or IP ranges, administrators create policies for individual applications or application groups. These policies can include conditions such as user identity, device condition, location, and time of access. ZPA enables internal applications like HR systems, development servers, internal knowledge bases, and databases to be separated with unique access rules. This type of **micro-segmentation** ensures that a user who has permission for one application cannot view or access other applications.

ZPA Architecture and Components

The architecture of ZPA consists of several important components that work together to deliver secure and scalable access to private applications.

ZPA Architecture and Components

1. **Zscaler Cloud (Service Edge):**
   A globally distributed cloud infrastructure that functions as an intermediary between users and applications. It manages authentication, applies security policies, and establishes connections without directly handling the application data traffic.

2. **App Connectors:**
   Lightweight virtual machines or container-based components deployed in private data centers or cloud environments. These connectors initiate outbound connections to the Zscaler cloud, removing the need for inbound firewall configurations. They create secure tunnels to the ZPA service edge and route traffic to internal applications.

3. **ZPA Client Connector:**
   A software application installed on end-user devices such as Windows, macOS, Linux, iOS, and Android systems. It verifies the user's identity, checks device compliance, and creates a secure micro-tunnel to the ZPA service edge for accessing applications.

4. **Application Segments:**
   Logical collections of applications configured in the ZPA administrative console. These segments represent private resources that authorized users can access.

Implementation Methodology

The implementation of ZPA follows a structured and phased deployment strategy to minimize disruption while maximizing security.

Phase 1: Discovery and Inventory
All internal applications currently in use are identified and classified based on factors such as importance, department usage, and user groups. Tools like network scanners, configuration management databases (CMDB), and discussions with department leaders help create a detailed application inventory.

Phase 2: Connector Deployment
ZPA App Connectors are installed in the environments where applications are hosted, including data centers and cloud platforms. This involves setting up virtual machines using platforms such as VMware, Hyper-V, or cloud images and configuring them to connect with the ZPA cloud.

Phase 3: Application Onboarding
Application segments are defined in the ZPA console by specifying the IP addresses and ports of internal applications. These applications are then linked with the appropriate App Connector groups.

Phase 4: Policy Creation
ZPA is integrated with the organization's identity provider, such as Azure Active Directory. Detailed access policies are created based on user groups, device security status, and geographic location. These policies are first tested in simulation mode before being enforced.

Phase 5: Pilot and Rollout
A selected group of users is chosen to test the new access system. Feedback is collected, performance is monitored, and policies are adjusted if necessary. After successful testing, additional users are gradually transitioned from traditional VPN access to ZPA.

Phase 6: VPN Decommissioning
After all applications and users have migrated successfully, the older VPN infrastructure is removed, reducing potential security vulnerabilities.

Tools and Technologies

- **Primary Platform:** Zscaler Private Access (ZPA), which acts as the main Zero Trust access solution.
- **Identity Provider (IdP):** Services like Microsoft Azure Active Directory, Okta, or Ping Identity for authentication and synchronization of user groups.
- **Endpoint Protection Platform (EPP):** Security tools such as CrowdStrike, SentinelOne, or Microsoft Defender that evaluate device security status before allowing access.

- **Security Information and Event Management (SIEM):** Platforms like IBM QRadar, Splunk, or LogRhythm used for centralized logging and monitoring of access activities.
- **Virtualization Platforms:** Technologies like VMware vSphere, Microsoft Hyper-V, or public cloud services such as AWS, Azure, and GCP used to host App Connectors.
- **Infrastructure as Code (IaC):** Automation tools such as Terraform or Ansible used for scalable deployment and configuration of connectors.
- **Digital Experience Monitoring (DEM):** Solutions that monitor user connectivity quality and application performance.

Integration with Identity Providers

Connecting ZPA with the organization's identity provider is essential for implementing identity-based access control. This integration typically uses protocols such as **SAML** or **LDAP**. Once integrated, ZPA synchronizes user attributes and group memberships so that access policies can directly reference these groups. When users log in through the ZPA Client Connector, they are redirected to the organization's identity provider for authentication and multi-factor verification. After successful authentication, identity information is sent back to ZPA, which evaluates the relevant access policies and determines which applications the user is permitted to access.

Device Posture Checking

In addition to verifying user identity, ZPA can enforce access policies based on the security condition of the user's device. By integrating with endpoint detection and response (EDR) or mobile device management (MDM) systems, ZPA can evaluate several security factors, including:

- Status and updates of antivirus software
- Disk encryption status such as BitLocker or FileVault
- Operating system patch level
- Detection of jailbroken or rooted mobile devices

- Risk scores provided by endpoint security tools

If a device does not meet the required security standards, ZPA can block the connection, allow limited access, or initiate a remediation process.

Micro-Tunneling and Application Visibility

ZPA establishes encrypted TLS or DTLS tunnels between the Client Connector and the ZPA service edge, and another secure connection between the service edge and the App Connector. This setup creates a direct micro-tunnel between the user and the application. The user's device is never placed on the corporate network, and traffic is handled at the application layer (Layer 7). As a result, users can only access the specific applications they are authorized to use, while all other internal resources remain hidden, preventing scanning, reconnaissance, and lateral movement attacks.

Threat Intelligence Integration

ZPA can also be connected with threat intelligence platforms to improve security monitoring. Activity logs generated by ZPA, including login attempts, policy violations, and user actions, can be forwarded to SIEM systems. These logs can then be analyzed alongside threat intelligence feeds to detect suspicious behavior. For example, if a user attempts to access an application from an IP address linked to known malicious activity, the SIEM system can trigger an alert and automatically revoke the user's session through integration with ZPA.

Incident Response in a ZPA Environment

ZPA improves the efficiency of incident response by providing several capabilities:

1. **Isolation:** Security teams can immediately revoke access for compromised users or devices through the ZPA console.
2. **Forensics:** Detailed logs show which applications were accessed, by whom, from where, and for how long.

3. **Containment:** Because users do not have network-level access, compromised endpoints cannot attack other systems.
4. **Granular Blocking:** Specific high-risk applications can be temporarily blocked while allowing access to others, ensuring business continuity.

Understanding ZPA: Key Takeaways

Zscaler Private Access represents a modern approach to secure remote access. Its main characteristics include:

1. **Cloud-native architecture:** Delivered entirely through the Zscaler cloud, removing the need for hardware appliances and complex VPN infrastructure.
2. **Identity-based access control:** Policies are created based on users and groups instead of IP addresses.
3. **Network invisibility:** Internal systems remain hidden from the internet, reducing exposure to attacks.
4. **High scalability:** The cloud architecture allows the platform to support large numbers of users simultaneously.
5. **Hybrid compatibility:** Works for applications hosted in on-premises data centers, private clouds, and public clouds.
6. **Improved user experience:** Users connect directly to applications without routing traffic through centralized VPN gateways, reducing latency.

Conclusion

Stage 1

Web application testing is an essential process for ensuring the functionality, reliability, and security of web-based software systems. It involves evaluating different components such as the user interface, system functionality, performance, and security to detect and resolve potential issues before they affect users. This process improves the overall quality of web applications and strengthens their resistance to cyber threats. As online platforms continue to

play a significant role in everyday life, thorough testing is necessary to deliver a secure and smooth user experience.

Stage 2

The Nessus security assessment report for the selected website provides valuable insights into its security condition. It identifies various vulnerabilities and classifies them based on severity levels, highlighting critical and high-risk issues that require immediate resolution. The report also provides recommendations such as applying patches, modifying configurations, and implementing security improvements. It emphasizes adherence to industry standards and best practices, including OWASP guidelines. Overall, the report stresses the importance of continuous monitoring, timely updates, and proactive security management to maintain a strong defensive posture.

Stage 3

The deployment of Zscaler Private Access represents a major transition from a traditional network-focused security approach to a modern identity-driven Zero Trust model. By applying least-privilege access and keeping private applications hidden from the internet, this solution effectively reduces the risk of lateral movement and network-based attacks. It enhances the organization's defense against evolving cyber threats while also providing a seamless and efficient experience for remote users. Additionally, separating access control from the network simplifies IT management, reduces reliance on legacy VPN systems, and offers detailed visibility into user-application interactions. Overall, this implementation creates a strong foundation for secure digital transformation in the future.

Future Scope

Stage 1: Future Scope of Web Application Testing

The future of web application testing is expanding rapidly due to the continuous growth of internet services and web-based platforms. As organizations increasingly rely on online systems, the importance of testing web applications will continue to grow.

Some important areas that highlight the future scope include:

- **Growing Demand:** As businesses shift their operations and services to digital platforms, the need for secure and reliable web applications will

increase. This will also raise the demand for effective web application testing to maintain performance, security, and functionality.

- **Emerging Technologies:** Modern web applications are becoming more complex because of technologies such as Artificial Intelligence (AI), Internet of Things (IoT), and blockchain. Testing these advanced systems will require new tools, techniques, and skilled professionals.
- **Focus on Security:** With the rise in cyberattacks and data breaches, security testing will become even more important. Testers will need to identify vulnerabilities, protect sensitive data, and ensure compliance with regulations such as GDPR.
- **Mobile and Cross-Browser Compatibility:** Since users access applications through multiple devices and browsers, ensuring compatibility across platforms will be essential. Testing must confirm that applications function properly on smartphones, tablets, and different web browsers.
- **Performance Testing:** As users expect fast and smooth application performance, techniques such as load testing and stress testing will become increasingly important to ensure applications can handle high traffic and heavy workloads.

Stage 2: Future Scope of the Testing Process

The future of the web application testing process, especially for detecting vulnerabilities, is highly promising. The rapid expansion of digital platforms and the growing importance of cybersecurity are driving advancements in testing techniques.

Key future developments include:

- **AI-Based Testing:** The use of Artificial Intelligence and Machine Learning in testing tools will transform vulnerability detection. These technologies can analyze large volumes of code and data quickly to identify potential security flaws that may be missed by manual testing.
- **IoT and Mobile Application Testing:** As IoT devices and mobile applications become more common, testing processes will expand to

cover these areas. Ensuring the security of IoT systems and mobile interfaces will become increasingly important.

- **Blockchain and Cryptocurrency Security:** With the growing adoption of blockchain technology and cryptocurrencies, security testing will be necessary to protect digital transactions and assets. Testing blockchain-based applications will become a specialized area within web application testing.
- **Serverless Architecture Testing:** As serverless computing becomes more widely used, testing will need to evaluate the security of serverless functions and APIs. Ensuring these components are secure and free from vulnerabilities will be critical.
- **Container Security:** With container technologies such as Docker and Kubernetes becoming widely used, testing will also focus on container security. This includes verifying that container images and orchestration configurations do not contain vulnerabilities.

Stage 3: Future Scope of Zero Trust

The successful implementation of Zscaler Private Access (ZPA) creates opportunities to expand the organization's overall security framework. Future improvements can further strengthen the security infrastructure.

- **Integration with Zscaler Internet Access (ZIA):** Combining ZPA with ZIA can create a unified Secure Access Service Edge (SASE) architecture that manages both internet and private application traffic through a single policy system.
- **Enhanced Device Security Checks:** Device posture verification can be improved by incorporating real-time risk scores from endpoint detection tools, allowing access to be blocked if a device shows signs of compromise.
- **Privileged Remote Access:** ZPA can be extended to provide secure and time-limited access for third-party vendors and contractors, replacing traditional jump servers and bastion hosts.

- **IoT and OT Segmentation:** Zero Trust principles can be applied to IoT and Operational Technology devices so that they communicate only with authorized management servers.
- **AI-Based Anomaly Detection:** Machine learning techniques can analyze ZPA logs to detect unusual user behavior, such as accessing applications at unusual times or from unfamiliar locations, triggering additional authentication or access restrictions.
- **Cloud Workload Protection:** ZPA can secure communication between containers and microservices in multi-cloud environments such as AWS, Azure, and GCP, ensuring that traffic between workloads is encrypted and authenticated.
- **Automated Access Management:** Integration with identity providers can automatically grant or revoke access when employees join, leave, or change roles, ensuring real-time access control.
- **Digital Experience Monitoring (DEM):** Implementing DEM tools can help monitor user connectivity and application performance, allowing organizations to identify issues such as latency before they affect productivity.

Topics Explored

1. Zscaler
2. Zero Trust
3. ZPA
4. VPN
5. MISP
6. College Network Information
7. Deployment of SOC in the College
8. Threat Intelligence
9. Incident Response

Tools Explored

- **Nessus Scanner**